

ENQUÊTES

De jour comme de nuit. Rendre compte des continuités du travail gratuit et rémunéré à partir des traces de l'activité informatique

Day and night. Accounting for the continuities of free and paid work through digital traces

Gabriel, Alcaras

chercheur post-doctoral au médialab (Sciences Po Paris) ; gabriel.alcaras@sciencespo.fr

Résumé

Cet article avance l'hypothèse que le travail gratuit et le travail rémunéré ne sont bien souvent que les deux faces d'un même processus productif. À partir d'une enquête sur la production d'un logiciel central dans nos infrastructures numériques, il analyse les rapports entre l'activité principale rémunérée – le « day job » – et les contributions bénévoles, effectuées en dehors des horaires de travail – « after hours ». *Grâce à des données originales couvrant près de 15 ans d'activité, cette recherche apporte un éclairage inédit sur la dimension temporelle de l'articulation entre travail gratuit et rémunéré.* L'article montre notamment que si la majorité des ingénieurs travaillent gratuitement, la minorité la plus active est désormais rémunérée par de grandes entreprises de la tech, dessinant des carrières continues du bénévolat vers le salariat. De plus, en analysant les rythmes hebdomadaires et quotidiens des contributions gratuites, l'article met en évidence que le travail gratuit n'est pas cantonné au temps libre, mais s'effectue aussi pendant les horaires de travail rémunéré.

Abstract

This article puts forward the hypothesis that free work and paid work are often just two sides of the same productive process. Based on a survey of software production central to our digital infrastructures, it analyzes the relationship between the main paid activity - the "day job" - and voluntary contributions made outside working hours - "after hours." Thanks to original data covering almost 15 years of activity, this research sheds new light on the temporal dimension of the articulation between free and paid work. In particular, the article shows that while most engineers work for free, the most active minority are now paid by major tech companies, tracing continuous careers from voluntary to salaried work. Furthermore, by analyzing the weekly and daily rhythms of free contributions, the article highlights that free work is not confined to free time but also occurs during paid working hours.

Mots-clés : travail gratuit ; informatique ; programmation ; temporalité ; méthodes mixtes ; données massives

Keywords : free work ; software ; programming ; temporality ; mixed methods ; big data

I'll have to work on this after work¹.

« Travailler après le travail ». En une formule laconique lâchée au détour d'un email, cet ingénieur résume comment s'effectue une large partie de la production informatique : chez soi, en dehors des horaires de travail et, surtout, gratuitement. Dans leurs discours, ces développeurs bénévoles opposent leur « *day job* », littéralement leur « travail de jour », au travail gratuit qui se déroule « *after hours* », après les heures de bureau. C'est une double vie qui se dessine, où le salariat de jour succède au bénévolat de nuit, comme si ces deux mondes ne se croisaient jamais. C'est pourtant le même mot – "travail" – qu'utilise cet ingénieur pour désigner ces deux moments supposément distincts. Comment rendre compte des continuités entre travail gratuit et rémunéré alors que ces deux types d'activité sont souvent cloisonnés ?

Dans l'informatique comme ailleurs, les activités considérées comme « hors travail » sont souvent invisibles et dévalorisées alors qu'elles contribuent massivement à nos économies. Le terme de « travail gratuit » permet de caractériser ces activités comme un « déni de travail » (Simonet, 2018) tout en ouvrant un dialogue avec les analyses sociologiques traditionnelles du travail. De nouveaux travaux viennent ainsi alimenter un corpus grandissant d'études sur le travail gratuit : le « travail à côté » des ouvriers (Weber, 1989) ; le travail gratuit des femmes dans la sphère domestique (Chadeau & Fouquet, 1981) et son rôle dans la reproduction des rapports sociaux de sexe (Blanchard et al., 2021) ; le recours au bénévolat, que ce soit chez les pompiers (Pudal, 2016) ou pour mettre au travail les allocataires de l'aide sociale (Simonet, 2020) ; l'exploitation du travail gratuit par les plateformes numériques (Broca, 2017) ; la disqualification du travail des personnes âgées (Derrien et al., 2023).

Le recours massif au travail gratuit dans l'industrie informatique, l'une des plus centrales et lucratives de nos économies, n'a donc rien d'étonnant en soi. 92 % des infrastructures numériques actuelles reposent sur des logiciels produits en grande partie grâce à du travail non rémunéré². En revanche, les spécificités du travail gratuit dans l'industrie informatique ont de quoi surprendre. Tout d'abord, ce travail gratuit s'est institutionnalisé à travers une forme spécifique – le logiciel libre. Ces logiciels, devenus indispensables au fonctionnement des infrastructures numériques, ne sont ni la propriété d'une entreprise, ni le résultat d'une planification quelconque ; ils sont gratuits et leur code est librement accessible sur internet. Les sciences sociales, en particulier l'anthropologie et la sociologie, ont consacré de nombreuses pages à cette institution particulière, en enquêtant auprès de collectifs (souvent bénévoles) qui s'organisent en ligne (Auray, 2003; Broca, 2013; Coleman, 2013; Demazière *et al.*, 2007; Kelty, 2008).

La seconde spécificité de cette forme de travail gratuit est qu'il relève plutôt d'un petit groupe d'hommes très qualifiés, au sein d'une profession déjà considérée comme une « élite » (Burrell & Fourcade, 2021), alors que le travail gratuit pèse habituellement sur des populations dominées ou marginalisées. Bien entendu, toute la profession ne peut pas être réduite à un même groupe homogène. Elle constitue de fait un espace particulièrement inégalitaire (Neely *et al.*, 2023), où les positions privilégiées de « la tech » (Alexandre, 2023) éclipsent souvent les emplois précaires ou

¹ « Je vais devoir travailler là-dessus après le travail » [1]. Dans la suite de l'article, les matériaux en ligne sont référencés par un index entre crochets, par exemple [4]. Ces références sont consignées dans l'annexe électronique jointe à cet article. Un index numérique comme « [4] » se réfère aux emails, tandis qu'un index alphabétique comme « [d] » fait référence à une contribution de code. Sauf mention contraire, toutes les citations issues de l'anglais (matériau de terrain, littérature scientifique) sont traduites par mes soins.

² « Open source is everywhere: survey results part 1 », Keenan Szulik, 12 avril 2018, URL : <https://blog.tidelift.com/open-source-is-everywhere-survey-results-part-1>

moyens (Bialski, 2024) dans le discours public. Mais l'industrie informatique reste dans l'ensemble un espace où les salaires et le niveau de diplôme sont plus élevés que dans la population active (revenu moyen annuel de 138 000\$ contre 65 470\$ aux États-Unis³ ; 39 495€ contre 21 610€ en France⁴). Quant à la réputation de l'informatique comme domaine masculin et masculinisé, celle-ci n'est plus à faire – à raison (Ensmenger, 2010; Hicks, 2017; Stevens, 2016).

Enfin, les logiciels libres ont cela de particulier d'avoir été traités presque exclusivement comme « hors travail » par les sciences sociales, loin des perspectives amenées par les études sur le travail gratuit⁵. La littérature existante sur l'informatique libre adhère en général à ce que Viviana Zelizer appelle une rhétorique des « mondes antagonistes » (Zelizer, 2001), c'est-à-dire le maintien d'une frontière étanche entre le don désintéressé et le monde de l'argent. Selon cette hypothèse, l'informatique libre serait ainsi parfaitement distincte de l'informatique marchande : « à de rares exceptions près, la production “libre” était dissociée du salariat [...]. Il s'agissait donc d'une activité indépendante de l'économie marchande », favorisant ainsi l'émergence « d'un autre monde numérique » qui se développerait « en marge de l'industrie des nouvelles technologies » (Broca, 2013, p. 71).

À rebours de cette opposition entre travail gratuit et rémunéré, cet article défend une hypothèse continuiste et montre que l'un prolonge l'autre. Il s'appuie sur une enquête consacrée à la production de Git⁶, un logiciel emblématique du travail gratuit dans l'industrie informatique. Ce logiciel a en effet la caractéristique d'être à la fois invisible et essentiel à la production informatique contemporaine. Peu connu du grand public, Git est hégémonique dans cette industrie : « aucune technologie n'est plus répandue que Git, surtout parmi les développeurs professionnels », qui sont 94 % à l'utiliser⁷. Ce logiciel, qui fait aujourd'hui partie du quotidien des développeurs, équipe les opérations liées à la logistique du code source, notamment pour contrôler et coordonner l'activité collective d'écriture. De l'auto-entrepreneur aux géants du numérique, la quasi-totalité des lignes de code produites dans le monde passent par Git. De plus, Git est un logiciel produit par un petit groupe d'ingénieurs, plus qualifiés et plus masculins que la moyenne de cette industrie, souvent employés par des grandes entreprises de la tech (Google, Microsoft, Facebook, Intel, etc.).

Le cas de Git a donc un double intérêt pour la compréhension du travail gratuit dans nos sociétés contemporaines. Il contribue d'une part à documenter comment le travail gratuit s'exprime dans les infrastructures informationnelles (Bowker *et al.*, 2010) : une activité considérée comme « hors travail », invisible (Star, 1999) et pourtant nécessaire à toute la production numérique contemporaine. D'autre part, il permet d'explorer les spécificités du travail gratuit dans l'industrie informatique, marquée par une institution particulière – le logiciel libre – et par les caractéristiques de ses travailleurs, une « élite » masculine et hautement qualifiée du code.

Afin de comprendre l'articulation entre travail gratuit et rémunéré, cette recherche s'appuie sur la complémentarité de jeux de données qui saisissent différents aspects de la production de Git, récoltés pendant une enquête menée entre 2017 et 2020. Sur le plan quantitatif, l'enquête mobilise

³ Estimations en 2023 d'après le U.S. Bureau of Labor Statistics, URL : https://www.bls.gov/oes/2023/may/oes_nat.htm#00-0000

⁴ Revenu des ingénieurs informatique estimé par le site d'annonces talent.com en 2024 ; revenu annuel moyen en 2023 en France d'après l'INSEE. Voir : « France, Portrait Social. Édition 2023 », *Insee Référence*.

⁵ Exception faite des travaux de Didier Demazière, François Horn et Marc Zune (Demazière *et al.*, 2007).

⁶ Les ingénieur-es prononcent « guite ».

⁷ « 2022 Developer Survey », StackOverflow, 2022, URL : <https://survey.stackoverflow.co/2022/#section-version-control-version-control-systems>

un vaste ensemble de données, qui retrace de manière exhaustive toute l'activité depuis la création de Git en 2005 jusqu'à la fin de l'enquête, en 2020. Au total, ces traces contiennent plus de 400 000 emails échangés sur la liste de discussion et environ 60 000 contributions de code effectuées sur le projet Git. Elles permettent de reconstituer l'activité de chacun des 1700 contributeurs⁸ à la ligne de code près, de mesurer leur niveau d'activité sur le temps long de la carrière ou le temps court des rythmes hebdomadaires. Ces contributeurs, qui sont principalement des hommes issus de pays du Nord (39 % d'Amérique du Nord et 36 % d'Europe) travaillant dans l'industrie informatique⁹, ne se sont pour la plupart jamais rencontrés physiquement, puisque la collaboration s'effectue presque exclusivement en ligne. Sur le plan qualitatif, l'enquête inclut des observations en ligne via la liste de discussion électronique, ainsi que des observations hors ligne menées lors de 11 conférences professionnelles entre 2017 et 2019 dans différents pays (France, Angleterre, Belgique, Espagne et États-Unis). Ces conférences sont les rares occasions où les contributeurs les plus actifs se retrouvent physiquement le temps d'un ou deux jours. De plus, 25 entretiens semi-directifs ont été réalisés avec des ingénieurs impliqués dans l'écosystème du projet Git pour retracer l'évolution conjointe de leurs contributions au logiciel libre et leurs mobilités professionnelles dans l'industrie informatique. Pour la plupart, ces entretiens se sont tenus sur le lieu de travail ou en marge des conférences, avec des ingénieurs issus du petit noyau de contributeurs les plus actifs, à la fois les plus présents lors des événements et les plus à même de répondre à des demandes d'entretien. Grâce à ces données originales, cette recherche apporte un éclairage inédit sur la dimension temporelle de l'articulation entre travail gratuit et rémunéré, notamment à l'échelle des horaires d'activité, pour mieux visualiser la continuité entre le travail de jour (« *day job* ») et de nuit (« *after hours* »).

Nous commencerons par exposer ce que nous pouvons apprendre sur le travail informatique en enquêtant à partir de ces traces. Dans un second temps, nous nous intéresserons à l'évolution et aux transformations du rapport entre travail gratuit et travail rémunéré au sein de Git. Enfin, nous porterons une attention particulière au travail gratuit, et à son articulation avec l'emploi principal rémunéré.

Penser le travail informatique à partir de ses traces

L'activité d'écriture informatique, c'est-à-dire la manufacture des codes sources, a cela de particulier qu'elle laisse un grand nombre de traces, archivées dans ce que les ingénieurs appellent des « dépôts de code » (Couture, 2012). Ces traces sont incroyablement riches et détaillées : il est possible de connaître l'auteur de chaque ligne de code, la date et l'heure à laquelle il l'a écrite, un message contextuel qui renseigne pourquoi il l'a écrite, ainsi que toutes les versions précédentes de cette même ligne depuis sa création. Les ingénieurs maintiennent ces archives non pas par souci de la postérité, mais tout simplement parce qu'elles permettent de coordonner plus efficacement l'écriture logicielle. Ces dépôts présentent une opportunité unique pour la sociologie du travail, mais viennent avec l'écueil classique des données massives (Ollion, 2016), à savoir un manque

⁸ Malgré la présence de quelques femmes parmi ces 1700 individus, nous utilisons le masculin générique tout au long du manuscrit. Ce choix a pour but de refléter l'hégémonie masculine à la fois dans la fabrication du logiciel, puisque nous estimons qu'il y a seulement 1 % de contributrices, mais aussi dans notre enquête, car les femmes sont encore plus rares dans le petit noyau actif qui se rend aux conférences auxquelles nous avons pu assister.

⁹ Ces estimations ont été obtenues suivant la méthodologie décrite dans la sous-partie « Enrichir les traces de l'activité informatique ».

presque total d'informations utiles pour cette discipline, à commencer par les caractéristiques sociales des ingénieurs. Dans cette perspective, Git est un logiciel comme un autre : s'il est possible de reconstituer l'histoire de chaque ligne de code ou la production de chaque contributeur, les traces de cette activité ne disent rien de son caractère gratuit ou rémunéré, sans parler du statut professionnel.

Cette première partie propose de penser cette indifférenciation non pas comme un obstacle méthodologique, mais comme le résultat d'un processus d'effacement des conditions de travail. Nous commençons par présenter les traces du développement de Git, leurs richesses et leurs lacunes. Nous présentons ensuite les processus d'effacement qui mènent à ces lacunes, avant de proposer une solution pour enrichir ces traces afin de mieux comprendre l'articulation du travail gratuit et rémunéré.

Des conditions de travail introuvables dans un déluge de données

Les traces du développement de Git sont principalement issues de deux infrastructures logicielles que les contributeurs utilisent pour mener à bien leurs tâches quotidiennes : d'une part, le dépôt du code source en tant que tel, qui contient toutes les contributions de code acceptées ; d'autre part, une liste de discussion publique, où ont lieu les échanges et les commentaires autour des contributions. C'est dans ces deux infrastructures, librement accessibles en ligne et employées depuis la création de Git en 2005, que se joue la majorité du travail collectif. Elles sont à la fois un espace où l'activité est en train de se faire et une archive qui conserve les traces du travail tel qu'il a été fait. Chacune de ses sources reflète différentes dimensions du travail.

Le dépôt du code source, surnommé `git.git`, saisit le fruit du développement, son aboutissement dans le logiciel final. Il retrace l'histoire de chaque contribution individuelle acceptée dans la version officielle qui, d'une certaine manière, n'est que la somme de ces 58 450 contributions. Ces dernières, appelées *commits*, représentent chacune un changement logique apporté au code source (modifier une fonction, ajouter de la documentation) accompagné de plusieurs métadonnées (la date, les lignes de codes modifiées, une signature individuelle ainsi qu'un message renseigné par le contributeur qui décrit le *commit*). Du fait de l'hégémonie de Git, le *commit* est aujourd'hui une catégorie commune de l'entendement de l'écriture informatique (Alcaras & Larribeau, 2022) et sert notamment à mesurer les performances individuelles. Les contributeurs emploient souvent cette métrique pour jauger leurs positions relatives : pour se présenter en entretien, un ingénieur me dit « si tu fais un `git shortlog`, je crois que je fais partie du top 12 [des plus grands contributeurs] ». Les *commits* mesurent donc le travail cristallisé dans le code final du logiciel.

Quant à la liste de discussion, ses 400 000 messages rendent compte des multiples itérations d'une même contribution, des propositions abandonnées ou refusées, bref, du travail en train de se faire. Pris dans leur ensemble, ces messages constituent une archive des activités discursives et des nombreux objets intermédiaires qui sont temporairement déployés avant de s'effacer (Vinck, 1999) devant la pérennité du dépôt de code. Le nombre de messages envoyés sur la liste de discussion capture donc l'activité de développement dans ses multiples dimensions, au-delà de la seule écriture du code.

La combinaison des traces issues du dépôt et de la liste de discussion aboutit ainsi à une archive presque exhaustive du travail collectif de développement, tout en identifiant bien l'activité de chaque individu. Les ennuis commencent dès qu'il faut relier ces traces de l'activité à leur contexte de production. Parmi les dizaines de milliers de *commits* enregistrés depuis 2005, une douzaine à peine comporte une référence directe à un emploi salarié. Trois *commits* seulement sont clairement décrits comme « sponsorisés » [a], dont un indique que Microsoft a fait don de son infrastructure de *cloud* pour automatiser une lourde série de tests [b], sans mentionner que l'auteur de la contribution est un employé de Microsoft. Bref, les messages de *commits* ne font presque jamais état de leurs conditions de production.

Des études tentent de contourner ce manque d'information, caractéristique des projets libres, en utilisant les adresses emails qui signent les *commits* comme variable de substitution. Elles postulent ainsi qu'un nom de domaine de corporation (@microsoft.com, @github.com) signifie « qu'un développeur est payé pour produire des logiciels libres et *open source* » (O'Neil *et al.*, 2022, p. 6). Cette variable de substitution, sûrement la seule praticable pour une analyse à grande échelle, présente néanmoins trois grandes limites. Premièrement, l'email est un mauvais indicateur du travail indépendant. En effet, les *freelancers* utilisent leur adresse personnelle pour signer leurs *commits*, occultant aussi bien la rémunération que la sujétion, parfois plus forte que dans le salariat, qui les lient aux sociétés clientes (Abdelnour & Bernard, 2018). Par exemple, un contributeur de Git à son compte se considère « presque comme un employé » de son principal client : il dispose d'un bureau dans l'*open space* parisien, traite régulièrement avec un employé qui « est un peu mon manager » et bénéficie même d'intéressements aux résultats. Deuxièmement, les adresses électroniques reflètent mal les changements de situation professionnelle car elles se trouvent au cœur d'une véritable économie de la signature, comme en science (Pontille, 2002). Puisque l'email sert à comptabiliser les *commits*, une majorité préfère éviter d'en changer, à l'instar des trois plus grands contributeurs de Git qui conservent leur adresse personnelle après leurs embauches par Google, GitHub et Microsoft. Troisièmement, l'usage de l'adresse professionnelle est moins un enjeu de rémunération que de propriété intellectuelle. Beaucoup de contrats stipulent que toute réalisation d'un employé, même accomplie sur leur temps libre ou sans recourir à du matériel de l'entreprise, appartient à l'employeur. Le règlement interne de Google stipule qu'« en général, Google détient les droits de propriété intellectuelle de tout développement effectué durant votre emploi »¹⁰ et, à ce titre, encourage les employés « à associer votre commit avec votre email google.com »¹¹. Un employé de Google pourrait ainsi signer une contribution de son email professionnel sans avoir été payé pour autant. L'adresse électronique n'est donc pas un indice suffisant et fiable des conditions de production, qui semblent ne pas laisser de traces.

La fabrique de deux mondes antagonistes

Comment expliquer que le statut des contributeurs soit à ce point effacé des traces du développement ? Une partie de cet effacement peut être attribuée aux infrastructures qui produisent ces dépôts, car elles imposent des catégories qui jouent parfaitement leur rôle de convention

¹⁰« Creating », Google Open Source, 4 janvier 2022, URL : https://opensource.google/documentation/reference/creating#personal_projects

¹¹« Open source patching », Google Open Source, 15 septembre 2022, URL : <https://opensource.google/documentation/reference/patching>

d'équivalence (Desrosières, 2001), établissant qu'une contribution gratuite en vaut bien une rémunérée. Mais ce processus d'effacement résulte également d'un travail constant que les ingénieurs fournissent pour maintenir une frontière étanche entre l'informatique libre et l'informatique marchande. La vision des « mondes antagonistes » s'obtient en effet grâce à une « saine gestion » des frontières (Zelizer, 2001, p. 123), qui prend soin de séparer les contributions au logiciel de leur contexte de production. Les contributeurs adhèrent d'autant plus à cette norme de la séparation qu'elle est liée à trois idéaux importants de la culture de l'informatique libre.

Le premier idéal est celui d'un espace public, incarné par la liste de discussion, laquelle a parfois été comparée à l'institution du café chez Habermas (Keltz, 2008, p. 8). À l'inverse, l'activité rémunérée et les conditions de travail sont perçues comme privées, d'où les formulations du type « sur une note personnelle », « c'est peut-être hors-sujet, mais ... ». En effet, ces informations s'échangent en général en privé, surtout lors des rares rencontres en face à face, par exemple lors du sommet annuel des contributeurs. Puisque la connaissance des statuts s'acquiert avant tout de façon interpersonnelle, elle est souvent lacunaire chez beaucoup de contributeurs :

Je dirais que sur les 20 principaux contributeurs historiques, je sais que 4 d'entre eux entrent dans la première catégorie [*plein temps payé*], 2 dans la seconde [*20 % de leur temps salarié consacré à Git*], et je sais que 3 autres sont affiliés à une entreprise (mais pas à quelle catégorie ils appartiennent). [...] Je ne connais pas la situation des autres, principalement parce que la question ne s'est jamais posée. [...] J'espère que ça répond correctement à ta question, même si c'est un peu vague parce que je n'ai pas tous les détails. [2]

Le second idéal est celui de la méritocratie du libre (Coleman, 2013, pp. 121-122), selon laquelle une contribution devrait être jugée sur ses qualités techniques, peu importe qu'elle soit l'œuvre d'un étudiant bénévole ou d'un ingénieur rémunéré. C'est au nom de ce principe que beaucoup affectent une forme de détachement vis-à-vis des situations professionnelles de leurs pairs, déclarant souvent « ce n'est pas important pour moi » ou encore « ça ne change rien », revendiquant parfois leur ignorance comme une vertu. Cet idéal méritocratique se nourrit d'un mythe fondateur de l'internet, issu de l'imaginaire politique des communes hippies : la possibilité de s'émanciper de tout marqueur physique ou social au sein de « communautés virtuelles » (Turner, 2008, pp. 159-161). C'est ainsi qu'à l'occasion d'une pause-café pendant une réunion de travail, un contributeur accueille avec bienveillance le *coming out* non-binaire de son collègue, avant de citer un *meme* célèbre — « de toute manière, sur l'internet, personne ne sait que tu es un chien » — pour immédiatement réaffirmer le voile d'ignorance que jetterait la collaboration en ligne sur les identités. Pourtant, le prétendu anonymat d'internet n'abolit pas les inégalités sociales, que ce soit pour les sites de rencontre (Bergström, 2016) ou les plateformes de micro-travail (Litman *et al.*, 2020). Dans le cas de l'informatique, les ingénieurs reconstituent l'identité d'un auteur de *commit* en se servant du nombre limité d'indices à leur disposition (Tsay *et al.*, 2014). Ainsi un pseudonyme perçu comme féminin augmente-t-il les chances qu'une contribution soit refusée si la personne n'a jamais participé au projet auparavant (Terrell *et al.*, 2017).

Enfin, les contributeurs séparent le libre du marchand au nom d'un idéal de l'autonomie du logiciel libre. Cette séparation constitue ainsi une stratégie professionnelle dans une lutte pour le contrôle légitime de leur expertise (Abbott, 1988). Les licences libres donnent aux ingénieurs les moyens d'extraire certains logiciels du portefeuille technique de leurs employeurs et, donc, de toute ingérence jugée indue. En retrouvant le contrôle sur leurs propres outils, les ingénieurs s'épargnent

la peine de réinventer le fil à couper le beurre dans chaque entreprise et affirment la primauté d'une certaine norme d'efficacité productive, proche de leurs routines de travail. En protégeant cette frontière hermétique, les ingénieurs cherchent moins à faire sécession qu'à « s'affranchir de l'informatique marchande pour mieux produire » (Alcaras, 2020).

Voilà pourquoi, lorsque le travail rémunéré est abordé, c'est le plus souvent pour rendre visible de potentiels conflits d'intérêt et réaffirmer, malgré la dépendance économique à une entreprise, l'autonomie des décisions techniques. Par exemple, lorsqu'il faut nommer des représentants officiels du projet dans un contexte associatif, les candidats détaillent le statut de leurs contributions : « je freelance depuis 2015 principalement pour Gitlab, Booking.com et Protocol Labs » [3] ; « je suis principalement un représentant de la catégorie des bénévoles, [mais] je suis également impliqué dans l'infrastructure Git de mon employeur [et] une partie du travail que je fais sur Git est sponsorisée » [4]. Quand une contribution est soupçonnée de ne bénéficier qu'à une entreprise, jouer carte sur table peut rétablir la confiance : « pour être parfaitement clair ici, cette fonctionnalité a été payée à 100 % par Github (ce qui ne veut pas dire que je ne pense pas que c'est une bonne idée. Au contraire, je pense qu'elle est géniale ; mais l'argent de GitHub est ce qui m'a donné le temps d'y travailler) » [5].

En définitive, les conditions de travail s'explicitent plutôt quand elles dévient de la norme du travail gratuit. Ce dernier apparaît souvent entre les lignes, sur l'air de « je vais essayer de m'y mettre dès que j'aurai un peu de temps libre » [6]. Cette norme de l'invisibilité ou, tout du moins, de la discrétion du travail gratuit s'explique lors d'un rare épisode de transgression. Contrairement à la coutume, un nouveau contributeur propose d'intégrer à la documentation publique de Git ce qu'il appelle une « supplique de relativement bon goût [*reasonably tasteful begging*] », que voici :

Des parts importantes [du logiciel] ont été réimplémentées par David Kastrup [...].
Merci de contribuer à sa rémunération si vous trouvez [ces améliorations] utiles. [7]

Ce à quoi le mainteneur de Git répond vertement : « merci, mais non merci — je ne trouve pas cela de bon goût [*tasteful*]. Faisons cela ailleurs » [8]. Le nouveau contributeur part alors en claquant la porte : « je ne compte certainement PAS investir plus de temps sur Git [puisqu'il] n'y a aucun moyen acceptable de faire prendre conscience aux utilisateurs de Git qu'il existe un lien entre une nécessité financière et le travail d'un *freelancer* comme moi-même » [9]. En somme, l'effacement des statuts et des conditions de travail ne résulte pas mécaniquement des infrastructures qui saisissent l'activité informatique : il est le fruit d'un effort collectif pour maintenir la séparation entre le libre et le marchand, pour des raisons à la fois idéologiques et stratégiques.

Enrichir les traces de l'activité informatique

Au vu de cette séparation, comment relier les contributions à leur contexte de production ? Ici, le recours à des sources externes s'impose puisque, comme nous l'avons vu, les traces de l'activité ne sont pas des proxy fiables. Ces sources externes sont principalement issues de l'enquête qualitative menée auprès des contributeurs (entretiens, observations) et de l'écologie informationnelle (Beaudouin, 2021) du projet Git, comme les profils sur les réseaux sociaux, les pages personnelles, etc. Étant donnée l'hétérogénéité de ces sources (Encadré 1), une approche automatisée n'est pas envisageable. Nous avons donc entrepris un codage manuel pour renseigner les différentes modalités d'emploi qu'occupe une personne tout au long de sa carrière de contributeur (entreprise ; titre de la position ; rémunération pleine, partielle ou gratuité du travail sur Git). Ses contributions

(commits du dépôt) et ses activités (messages de la liste) sont ensuite croisées avec la position professionnelle qu'occupe son auteur au moment où elles ont été effectuées. Ce codage manuel permet ainsi de contrôler avec exactitude ce que recouvrent les catégories de travail gratuit ou rémunéré.

Encadré 1. Coder le statut à partir de sources hétérogènes

Pour coder le statut et la profession, les sources d'information ont été hiérarchisées en privilégiant les sources les plus fiables.

1. Sources produites en interaction avec les contributeurs, en mobilisant entretiens, observations et échanges privés.
2. Sources en ligne liées au projet Git : messages de la liste de discussion, wikis, interviews données à la gazette mensuelle *Git Rev News* ou à certains podcasts spécialisés (*Git Minutes*, *All Things Git*).
3. Pages personnelles des contributeurs (CV en ligne, compte LinkedIn, sites et blogs personnels), en faisant fréquemment appel aux archives du web telles que Internet Archive.

En effet, certaines situations sont claires, aussi bien dans le sens du travail rémunéré (membre d'une « Git team » dans une grande société, contrat court pour développer une fonctionnalité précise, indépendant qui vend son expertise de développement Git, stagiaire rémunéré) que dans celui du travail gratuit (étudiant, chômage). Mais toutes les situations ne sont pas si clairement tranchées. Que faire d'un employé dont les contributions sont liées aux ressources de son employeur, par exemple pour adapter Git à un système propriétaire de serveurs à haute fréquence ? Peut-on considérer qu'un chercheur en physique quantique qui contribue régulièrement à Git durant ses horaires de bureau est payé par son Université pour ce travail ? Pour lever l'ambiguïté, nous avons fait le choix d'une approche déclarative : sauf mention directe et explicite de la part des personnes concernées que Git constitue tout ou partie de leur activité rémunérée, leurs contributions sont catégorisées comme relevant du travail gratuit. Ces cas de figure soulignent la continuité qui peut exister entre les contributions au logiciel libre et l'emploi principal.

Ce codage manuel a été effectué sur un échantillon représentatif des contributeurs, qui tient compte de « l'inégalité de participation » propre aux collectifs en ligne, selon lequel « quelques utilisateurs contribuent la majorité écrasante du contenu, tandis que la plupart ne postent que très rarement voire pas du tout » (Nielsen, 1998). L'inégalité de participation n'est pas « un principe explicatif en soi », mais elle constitue un constat qui doit alimenter la réflexion méthodologique et borner les efforts interprétatifs (Akrich, 2012, p. 27). Le développement de Git, comme tous les autres logiciels libres (Demazière *et al.*, 2007) n'échappe pas à cette caractéristique morphologique. L'inégalité de participation y est même particulièrement marquée, puisque 1 % seulement des 1731 contributeurs a produit plus des deux tiers du logiciel.

En quoi cette inégalité de participation affecte-t-elle notre choix d'échantillonnage ? Il y a fort à parier qu'une personne rémunérée pour travailler sur Git, ne serait-ce que quelques mois, aura davantage de contributions à son actif que la plupart de ses pairs. En particulier, l'hypothèse selon laquelle la quasi-totalité des personnes rémunérées ont effectué au moins 10 commits — soit davantage que les trois-quarts des contributeurs — est plus que raisonnable. En partant du principe que le travail gratuit est la norme dans ce collectif, nous avons donc plus de chance de trouver du travail rémunéré parmi les personnes qui ont écrit quelques commits que dans la longue traîne de très petits contributeurs. Pour ces raisons, l'algorithme d'échantillonnage privilégie les contributeurs les plus actifs (Encadré 1). L'échantillon final réunit 205 personnes, dont le travail cumulé représente 80 % de tous les commits effectués entre 2005 et 2020. Leur volume de contributions va de majeur (plusieurs milliers ou centaines de commits) à modeste (moins d'une dizaine). Le codage manuel a permis de renseigner les conditions de travail de 146 personnes (soit 71 % de l'échantillon). Le statut des contributions des 29 % restant sera qualifié "d'inconnu" dans la suite de l'analyse, même s'il est très probable qu'elles relèvent du travail gratuit.

Encadré 2. Algorithme d'échantillonnage

Pour chaque trimestre :

4. Obtenir une liste des contributeurs du trimestre grâce à la commande `git log`.
5. Ordonner cette liste par ordre décroissant de *commits*, de sorte que la personne qui a le plus contribué se trouve en haut de la liste et celles qui ont le moins contribué en bas.
6. Lorsque plusieurs personnes occupent le même rang dans cette liste, c'est-à-dire qu'elles ont exactement le même nombre de *commits*, l'ordre des contributeurs au sein de chaque rang est généré aléatoirement.
7. Sélectionner le plus petit nombre des n premiers contributeurs de la liste dont les commits cumulés dépassent 80 % des commits du trimestre. En moyenne, la dernière personne de cette sélection trimestrielle a écrit 12 commits ; jamais moins de 5 et 32 tout au plus.
8. Répéter pour le trimestre suivant.

Cette sonde trimestrielle prend en compte les contributeurs de passage, actifs pendant quelques mois, et les contributeurs de long terme au rythme contribution irrégulier.

Quant à la représentativité de cet échantillon – une notion polysémique, qui échappe aux tentatives d'en stabiliser un sens purement technique (Didier, 2013) –, ce dernier n'est pas représentatif au

sens aléatoire du terme, mais il représente une sous-population pertinente pour comprendre comment les proportions de travail gratuit et rémunéré évoluent au cours du temps. Il éclaire en effet une grande variété de niveaux de contributions et de situations professionnelles. Ainsi, même l’auteur d’une dizaine de commits, ce qui le situe de facto dans le top 20 % des contributeurs, peut déclarer en entretien « ne pas se définir comme un “vrai” contributeur » et ne pas le mentionner dans son CV en ligne. Cette situation s’observe également sur la liste de discussion : ici l’auteur d’une vingtaine de commits (top 12 %) se présente comme un « contributeur très occasionnel », là un contributeur qui compte 40 commits (top 7 %) à son actif déclare « je ne peux pas vraiment répondre à cette question, je ne suis qu’un petit contributeur ». L’inégalité de participation est telle que même les rares personnes qui cumulent une dizaine de commits se sentent plus proches de la masse des petits que de la poignée des gros contributeurs. Le sens émique du terme « contributeur » diffère ainsi de la signification qu’il a dans cet article, qui désigne toute personne qui a contribué au moins un commit. Cet échantillon comporte donc des personnes qui se perçoivent comme des contributeurs de moyenne ou de petite importance, voire qui ne se considèrent pas comme des « vrais » contributeurs.

En définitive, la solution que nous proposons pour contourner l’effacement des conditions de travail des traces de l’activité joue un double rôle. D’un côté, elle stabilise des catégories claires pour l’analyse quantitative : le statut d’une contribution est déterminé selon des critères déclaratifs (rémunérée si son auteur mentionne Git comme faisant partie de son emploi, gratuite sinon, inconnue quand aucune information sur l’emploi n’est disponible), tandis que les contributeurs sont caractérisés par des critères objectivés (avoir au moins un *commit* dans le dépôt officiel). D’un autre côté, le processus de codage et d’échantillonnage révèle des cas ambigus, ce qui souligne la difficulté de tracer une frontière nette entre travail gratuit et rémunéré, ce qui vient renforcer l’hypothèse d’une continuité entre ces deux formes de travail. La quantification joue ici son double rôle d’instrument de connaissance et d’objet discutable, qui ouvre un débat (Desrosières, 2008, pp. 77-100).

Contribuer au grand jour

La vision des « mondes antagonistes » invisibilise donc les statuts des contributeurs en les effaçant des traces de l’activité, ce qui renforce en retour la perception du logiciel libre comme un espace à part, détaché de la logique marchande. Pour pallier cet effacement, nous avons alors proposé d’enrichir ces données par une méthode d’échantillonnage. Dans cette deuxième partie, nous mettrons à profit ces données enrichies pour nous intéresser à la place qu’occupe réellement le travail gratuit dans le projet Git. Nous montrerons d’abord que, contrairement à une idée reçue, le travail rémunéré est loin d’être négligeable dans le logiciel libre, même si le travail gratuit reste la norme. Nous expliquerons ensuite pourquoi certaines entreprises sont prêtes à rémunérer, voire à embaucher des contributeurs qui travaillent déjà gratuitement. Enfin, nous reviendrons sur les trajectoires des contributeurs qui passent de bénévoles à salariés.

Le travail rémunéré, à la fois règle et exception

Le rapport entre travail rémunéré et travail gratuit s’estime sur deux dimensions : d’une part, vis-à-vis des individus qui contribuent au logiciel (Figure 1) et, d’autre part, vis-à-vis des contributions de

code versées à la version officielle de Git (Figure 2). L'analyse montre qu'aujourd'hui, une minorité de contributeurs rémunérés assure la majorité du développement. Au premier trimestre 2020, 10 % des 80 personnes dont les contributions ont été acceptées signent 60 % des 956 *commits* enregistrés. Nous retrouvons ici le phénomène de l'inégalité de participation : les contributeurs rémunérés sont, en général, ceux qui font le plus grand nombre de contributions.

La catégorie du travail « partiellement rémunéré » des figures 1 et 2 mérite que de s'y attarder brièvement. Cette dernière caractérise des arrangements officiels entre le contributeur et l'employeur, en général pour consacrer environ 20 % de son temps de travail rémunéré au développement libre. C'est par exemple le cas de Junio¹², qui convainc son employeur de financer, conjointement avec une entreprise japonaise, une journée hebdomadaire consacrée à Git :

Je tiens également à remercier Twin Sun Inc (mon employeur) et NEC d'avoir promis de me soutenir dans mon travail sur GIT à temps partiel à partir de maintenant. Je prévois de passer 8 à 12 heures par semaine dans le cadre de mon travail principal [*day job*] ; les soirées et les week-ends sont à ma charge, comme avant. Mon plan provisoire est de faire du mercredi et du samedi mes principaux jours de travail sur GIT. [10]

Puisque ces arrangements rendent impossible d'identifier si une même contribution a été réalisée au titre du travail rémunéré, gratuit ou d'une combinaison des deux, toutes les contributions effectuées sous ce statut sont caractérisées comme « partiellement rémunérées » dans les analyses qui suivent.

¹² L'anonymisation est une tâche délicate dans le cadre d'un terrain impossible à dissimuler (le logiciel Git existe véritablement) et où toute l'activité a lieu sur un espace de discussion public, perçu comme tel par tous les contributeurs. Nous avons pris la décision de garder les véritables prénoms lorsque nous citons ces prises de parole publique (liste de diffusion, messages de commit, etc.). En revanche, les extraits d'entretien et les statistiques agrégées emploient des prénoms alternatifs.

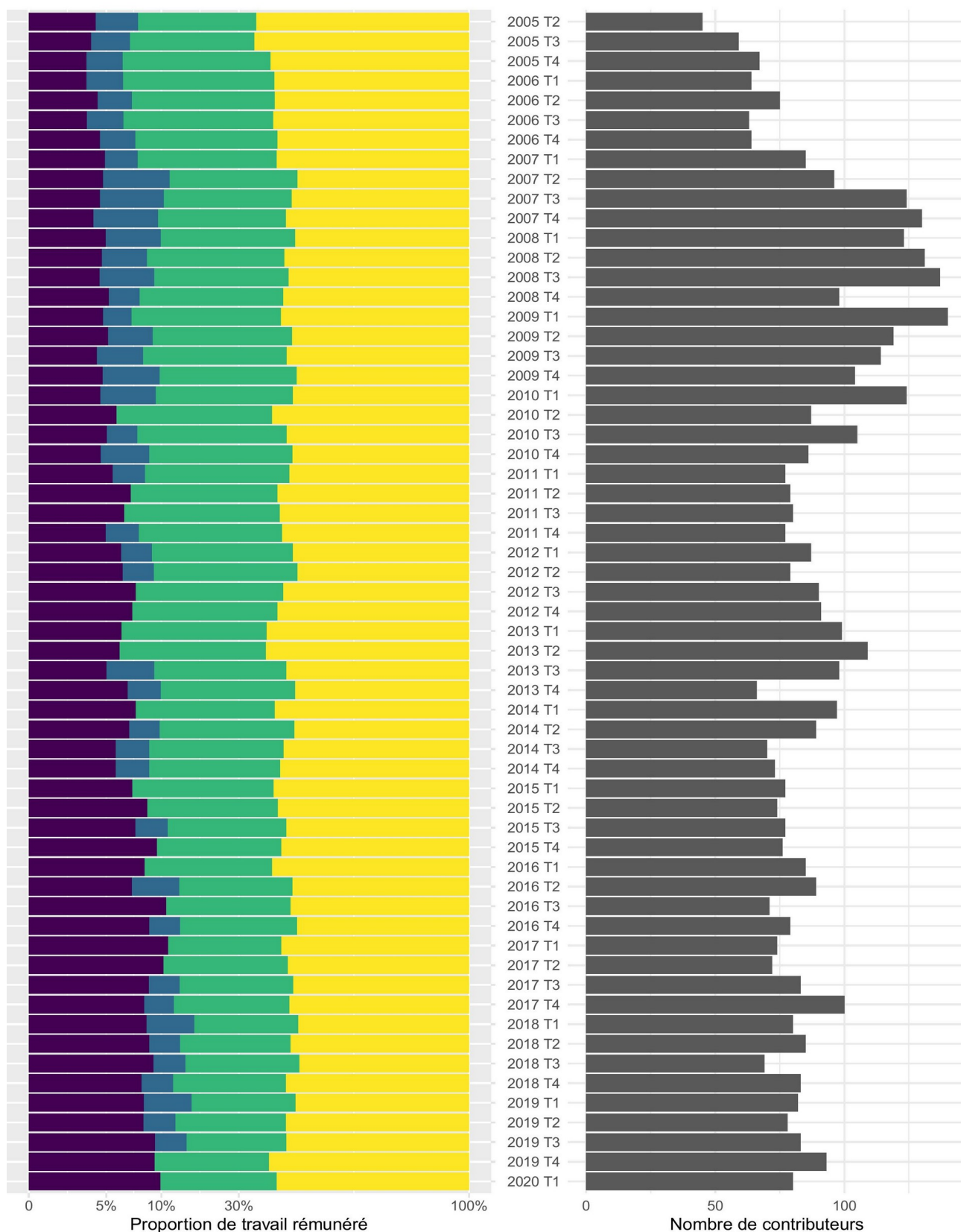


Figure 1. Les contributeurs rémunérés, une exception de moins en moins rare

Les deux diagrammes partagent la même échelle temporelle, au centre, graduée en trimestres (de T1 à T4). À droite, le diagramme indique le nombre de personnes qui ont contribué à Git ce trimestre. À gauche, le diagramme décrit la proportion de personnes dont le travail de contributions est pleinement rémunéré, partiellement rémunéré (le plus souvent une demi-journée à une journée) ou effectué gratuitement. Pour des raisons de lisibilité, le diagramme de gauche présente une échelle logarithmique, comme l'indique la graduation de l'axe des abscisses.

Lecture : parmi les 48 personnes qui ont contribué au deuxième trimestre 2005 (T2), environ 5 % ont été pleinement rémunérées pour ce travail, tandis que 5 % ont bénéficié d'un temps partiel de leur employeur. Le statut des autres contributeurs est indéterminé. **Champ :** 1750 personnes jusqu'au premier trimestre 2020 (inclus).

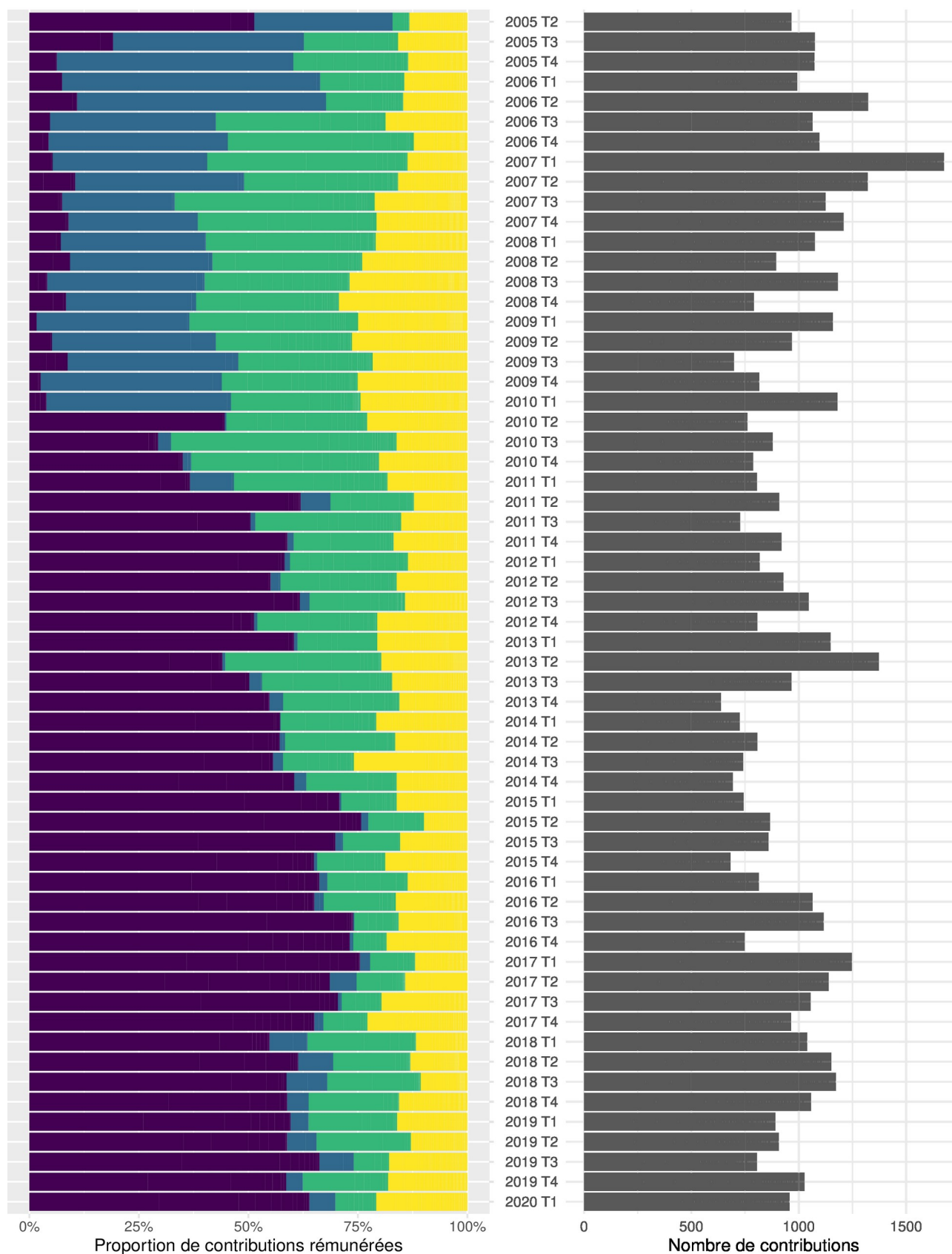


Figure 2. Les contributions rémunérées deviennent la règle

Les deux diagrammes partagent la même échelle temporelle, au centre, graduée en trimestres (de T1 à T4). À droite, le diagramme indique le nombre de contributions enregistrées sur la version officielle de Git. À gauche, le diagramme décrit la proportion de contributions effectuées par une personne pleinement rémunérée, partiellement rémunérée ou effectuées sans rémunération.

Lecture : parmi le millier de contributions du premier trimestre 2005, environ 50 % ont été produites dans le cadre d'un temps plein et 30 % dans le cas d'un temps partiel. Le statut des 20 % restants est indéterminé. **Champ :** 58 450 contributions (commits) enregistrées sur le dépôt Git jusqu'au premier trimestre 2020.

Ces graphiques révèlent également que le travail rémunéré alimente le développement de Git dès sa conception. Le projet émerge alors pour répondre aux besoins d'organisation d'un logiciel phare du libre, le noyau Linux. Une grande partie des premiers contributeurs de Git proviennent donc des fameux *kernel hackers*, à commencer par le créateur des deux projets, Linus Torvalds. En mai 2005, Linux existe depuis bientôt 14 ans : ce logiciel libre et une petite minorité de ses développeurs ont déjà trouvé leur place dans l'informatique marchande, surtout dans le domaine en pleine expansion de l'Internet et des serveurs. Git bénéficie ainsi des contributions des développeurs de Linux, y compris de ceux qui sont rémunérés à temps plein. Ces derniers représentent au mieux 5 % des contributeurs de Git mais cumulent, au départ, plus de la moitié des contributions, pour l'essentiel attribuées à Torvalds. Au terme de quelques mois de développement, ils se consacrent à nouveau à Linux, d'où une diminution rapide du travail rémunéré après le deuxième trimestre 2005.

Bien qu'être payé pour travailler sur Git ait toujours été une situation exceptionnelle, cette dernière est devenue de moins en moins rare. Les transformations s'accélèrent à partir de 2010. En cinq ans, une dizaine de personnes – en particulier trois contributeurs historiques, Junio, Jeff et Johannes – accèdent à des postes qui financent leurs contributions : une minorité d'entre elles en *freelance* ; une majorité grâce des emplois stables dans de grandes entreprises. Premier à décrocher un emploi à plein temps en 2010, Junio mentionne laconiquement son embauche au détour d'un message : « au fait, pour votre information, j'ai récemment pris un poste chez Google » [11]. Un an plus tard, Jeff rejoint une start-up californienne qui a le vent en poupe : « certains d'entre vous le savent peut-être déjà, mais j'ai commencé à travailler à plein temps pour GitHub en janvier [2011], spécifiquement pour travailler sur git » [12]. En 2015, Johannes, rejoint Microsoft et « le club des développeurs qui sont payés pour travailler sur Git dans le cadre de leur travail quotidien [*day job*] » [13]. Alors que moins de 5 % des contributeurs étaient rémunérés avant 2010, cette proportion avoisine et dépasse régulièrement les 10 % après 2015 : bien qu'elle fasse toujours figure d'exception, cette exception se fait de moins en moins rare. Surtout, pour la première fois depuis la création de Git, le travail rémunéré représente plus de la moitié des commits de chaque trimestre et devient la règle en matière de contributions.

Quand les entreprises investissent dans l'informatique libre

Comment expliquer que des entreprises soient prêtes à payer des ingénieurs qui travaillent déjà gratuitement ? Le premier intérêt est d'internaliser l'expertise des contributeurs. Cette dernière concerne évidemment la technologie, surtout dans le cas d'une matière complexe et précise comme la gestion de versions, mais aussi le processus de collaboration libre en tant que tel. Un contributeur expérimenté connaît les acteurs clés du projet et leurs spécialités respectives, a en mémoire l'histoire technique du code, maîtrise une organisation du travail très codifiée (rythmes de développement, division de l'activité) ainsi que des normes implicites (utilisation de la copie Cc dans les emails, réciprocité des évaluations de code). Enfin, un contributeur de longue date bénéficie d'une réputation de confiance, obtenue au fil des années au contact de ses pairs. Ce sont ces ressources, à la fois rares et longues à acquérir, pour lesquelles payent les entreprises.

Le deuxième enjeu de ces embauches est celui du contrôle. Le travail des bénévoles ne coûte rien aux entreprises, mais cette gratuité s'accompagne d'une liberté dans le choix des tâches exécutées. Le développement libre a beau être organisé et hiérarchisé (Demazière *et al.*, 2007), les contributeurs choisissent ce sur quoi ils travaillent – et leurs préoccupations individuelles ne sont pas celles des grandes entreprises. La subordination, caractéristique de la relation salariale, donne

aux entreprises un moyen de contrôler une partie du travail contributif. Les contributeurs salariés ne deviennent pas de simples exécutants du jour au lendemain. Au contraire, ce contrôle va s'exercer dans le respect des normes collaboratives que les contributeurs salariés connaissent bien, ce qui le rend d'autant plus efficace. Lors d'un entretien, un contributeur salarié me confirme qu'il reçoit dès son embauche une liste de fonctionnalités à développer, qu'il présente comme des « problèmes difficiles sur lesquels je n'aurais pas travaillé par moi-même », tout en continuant à s'occuper de tâches qui ne bénéficient pas directement à son employeur. Outre la subordination, le contrôle a également une nature juridique puisque, sauf renégociation du contrat de travail, l'entreprise est propriétaire des contributions de son employé. Cet aspect du contrôle, qui semble plutôt être une conséquence de l'embauche que son motif premier, s'inscrit dans le cadre de la construction de véritables monopoles intellectuels (Rikap & Lundvall, 2020).

Rémunérer les contributeurs peut enfin relever d'un intérêt au désintéressement. Pour certaines entreprises très riches, l'embauche d'un contributeur peut faire partie d'une stratégie plus large de mécénat (Depecker *et al.*, 2018). Google est la première entreprise à embaucher un contributeur bénévole de Git en 2010, le mainteneur du projet. Le rôle de ce dernier n'est pas de développer de nouvelles fonctionnalités mais de veiller à ce que les contributions s'intègrent de manière harmonieuse dans le logiciel final. Puisque le mainteneur a le dernier mot sur ce qui sera intégré au code, son pouvoir d'arbitre est considérable et sa réputation d'impartialité capitale pour asseoir sa légitimité. Autant que l'enquête puisse en juger, il semblerait que Google ait effectivement garanti l'indépendance du mainteneur en le séparant hiérarchiquement et physiquement de ses autres contributeurs salariés. Ces entreprises se présentent ainsi comme de « bonnes citoyennes » (*good citizens*) de l'informatique libre qui « rendent à la communauté » (*give back to the community*) afin de faire preuve de leur bonne foi (Simonet, 2002). Au-delà de construire une image positive dans ce milieu professionnel, ces bénéfices réputationnels servent les entreprises lorsqu'elles décident d'intervenir dans le monde de l'informatique libre.

Le logiciel libre, de l'*after hours* au *day job*

Dans ce contexte d'investissements visibles et massifs de la part de grandes entreprises, l'informatique libre est souvent caractérisée comme la cible passive de stratégies commerciales : le logiciel libre serait ainsi « coopté, récupéré et trahi par les mastodontes de l'industrie »¹³. Or ce discours ne tient pas compte des trajectoires des bénévoles qui deviennent salariés de ces grandes entreprises. Pour beaucoup d'entre eux, faire du développement libre leur activité principale constitue, sinon l'objectif conscient d'une stratégie professionnelle, parfois un rêve inaccessible, toujours une reconnaissance de la valeur de leur travail et de leur savoir-faire.

Il est vrai que certains développeurs sont les premiers surpris de se retrouver employés de grandes entreprises qu'ils considéraient autrefois comme des ennemies du libre : « si tu m'avais demandé, dix ans plus tôt, si je pourrais un jour travailler à Microsoft, je t'aurais directement ri au nez », me dit un ingénieur en entretien. Ces contributeurs présentent rarement leurs embauches comme le résultat d'une évolution personnelle qui, au fil des années, les pousserait à revoir des positions de jeunesse radicales et idéalistes. Au contraire, ils mettent en avant les transformations de ces entreprises qui, comme Microsoft, donnent désormais des gages concrets de soutien à

¹³« Le pillage de la communauté des logiciels libres », Mathieu O'Neil, Laure Muselli, Fred Pailler & Stefano Zacchiroli, *Le Monde diplomatique*, Janvier 2022, URL : https://www.monde-diplomatique.fr/2022/01/O_NEIL/64221

l'informatique libre, notamment en recrutant un contributeur de longue date. Dans leur récit biographique, l'embauche est vécue comme une continuité plutôt qu'une rupture : « après avoir maintenu Git for Windows sur mon temps libre pendant près de huit ans, j'ai maintenant le plaisir d'être payé pour le faire » [13].

À l'inverse de ceux qui ne pensaient jamais voir l'industrie informatique se transformer au point de les embaucher, d'autres bénévoles entretiennent le doux rêve d'être rémunérés pour leurs contributions. Pour ces ingénieurs, la position de contributeur salarié n'est pas l'objectif d'une stratégie professionnelle consciente, souvent parce qu'ils ne savent pas comment l'atteindre. De fait, ces postes passent rarement par le processus standardisé de recrutement (annonces officielles, envoi de CV) et dépendent souvent de relations interpersonnelles. Ces contributeurs acceptent alors la norme du développement *open source*, gagnent leur vie grâce à un *day job* alimentaire et contribuent pendant leur temps libre ou lors de quelques heures grappillées au travail. C'est le cas de Tom, qui raconte son parcours en entretien :

En fait, j'ai fini par abandonner mes études, je n'ai jamais eu mon diplôme [rires], mais... Et j'ai eu des enfants entretemps, j'ai eu quelques jobs ici et là, donc il y a eu des hauts et des bas, c'est clair. Mais j'avais l'impression que [l'*open source*] faisait partie intégrante de mon identité. Donc, après la fac, j'ai fait un peu de sous-traitance pour du logiciel propriétaire, j'avais trouvé un poste [...] comme administrateur système, ça me prenait 15 heures par semaine et je pouvais passer le reste du temps à travailler sur du libre. Je m'étais un peu engagé dans ce style de vie, en me disant « ok, c'est une façon durable de faire de l'*open source* pour les 20 prochaines années », et c'était important pour moi. [...] Bon, tu vois, je n'étais pas très content de ne pas gagner beaucoup d'argent, mais je me disais que si je pouvais faire ça, être heureux et gagner assez d'argent, ce serait très bien, donc je me suis dit ok. Donc, ouais, et je, plusieurs fois, je me souviens que plusieurs fois j'en avais parlé avec ma femme, tu vois : « ah, j'adorerais trouver un job pour faire de l'*open source*. Bon, je sais que ça existe, mais comment est-ce que les gens font pour trouver ces postes ? ».

Ce rapport au travail gratuit est celui d'un « rêve rationnel », loisir légitime qui pourrait devenir rentable sans être explicitement formulé comme tel, typique des classes supérieures, que Florence Weber oppose au « travail à-côté » des ouvriers, moyen de compensation fondé sur la subsistance (Weber, 1986, p. 22). Cette caractérisation apporte un éclairage complémentaire au portrait classique du contributeur bénévole comme *hobbyiste* (Himanen, 2001), dont la principale motivation serait la poursuite de son plaisir, « *just for fun* »¹⁴, mais dont les productions gratuites pourraient lui gagner la reconnaissance de ses pairs et des institutions marchandes.

Enfin, pour quelques rares contributeurs, l'embauche est un objectif conscient qui s'intègre dans une stratégie professionnelle. Ces bénévoles, qui veulent gagner leur vie grâce à leurs contributions, n'attendent pas d'être repérés par une grande entreprise : ils sont prêts à renoncer aux protections sociales et à la stabilité d'un emploi salarié pour se tourner vers le travail indépendant. Forts de leurs connaissances expertes sur le logiciel, ils peuvent ainsi vendre leurs compétences à des entreprises pour développer des fonctionnalités spécifiques ou assurer du support technique. Pour ces contributeurs, l'embauche par une grande entreprise constitue tantôt une fin, tantôt un moyen. Dans le premier cas, le contributeur utilise ses missions de *freelance* pour nouer des relations avec des sociétés clientes qui pourraient, à terme, l'embaucher. Par exemple, un développeur a fini par

¹⁴ Selon l'expression consacrée issue de la biographie de Linus Torvalds. Voir : Torvalds L. & Diamond D. (2002), *Just for Fun: The Story of an Accidental Revolutionary*, New York, Harper Business.

être salarié par son principal client après plusieurs années en tant qu'indépendant. Dans le second cas, l'embauche par un géant du numérique s'intègre à une stratégie de moyen terme : après deux ans passés dans la Silicon Valley, il rentre en France avec une épargne solide avec laquelle il achète un appartement en banlieue parisienne et de créer sa propre société coopérative (SCOP) pour rémunérer son propre travail et celui d'autres contributeurs.

L'embauche, inattendue ou recherchée, a des conséquences majeures sur les conditions de travail des contributeurs. La première conséquence est, évidemment, d'ordre financier. La plupart des entreprises qui peuvent embaucher des contributeurs sont des géants de l'industrie numérique (Google, Microsoft, Facebook, Apple), des très grandes entreprises et des start-ups en pleine croissance. Les rémunérations sont d'autant plus confortables que les contributeurs sont souvent recrutés en tant qu'experts ou développeurs *senior*. L'embauche représente, pour beaucoup, une augmentation notable de leurs rémunérations, sans atteindre toutefois les sommes les plus élevées de la Silicon Valley.

La deuxième conséquence concerne l'activité de contribution elle-même. Comme le montre l'analyse des horaires de travail (Figure 3), l'embauche marque littéralement le passage d'une activité exercée *after hours* (en dehors des horaires de travail) à un *day job*, un travail de jour. L'effet le plus notable est la sanctuarisation du week-end : les contributeurs salariés réduisent drastiquement, voire arrêtent complètement de travailler le samedi et le dimanche. Le second effet est une diminution du travail de nuit (entre 22h et 5h du matin). Cette diminution est plus ou moins prononcée selon les préférences et les modalités d'emploi des contributeurs. En effet, ceux qui travaillent à distance ont des horaires moins clairement définis que ceux qui se rendent au bureau chaque jour.

L'analyse des horaires révèle deux autres éléments importants. Tout d'abord, ce changement du rythme de travail n'affecte pas le niveau d'activité, qui reste sensiblement le même avant et après l'embauche. Autrement dit, ces données ne soutiennent ni l'hypothèse d'un sur-investissement dans l'activité bénévole afin d'obtenir le statut de salarié, ni celle d'une mise au travail soudaine dans le cadre de la subordination salariale. Cette constance observée des développeurs, qui répond au sentiment déclaré d'une continuité entre activité bénévole et salariée, reflète un engagement dans le « vrai boulot » de la contribution (Bidet, 2011), qui n'est pas pratiquée comme le simple moyen de décrocher un poste bien payé. Un bénévole devenu salarié me raconte comment, après son embauche par Google, il est affecté à son grand dam au développement de la régie publicitaire et demande immédiatement un transfert pour travailler à plein temps sur Git. Un deuxième élément corrobore cet attachement à un logiciel spécifique : après leur embauche, les contributeurs ne s'investissent pas bénévolement dans un nouveau projet libre. Une fois que Git devient leur travail de jour, le travail de nuit s'efface pour de bon.

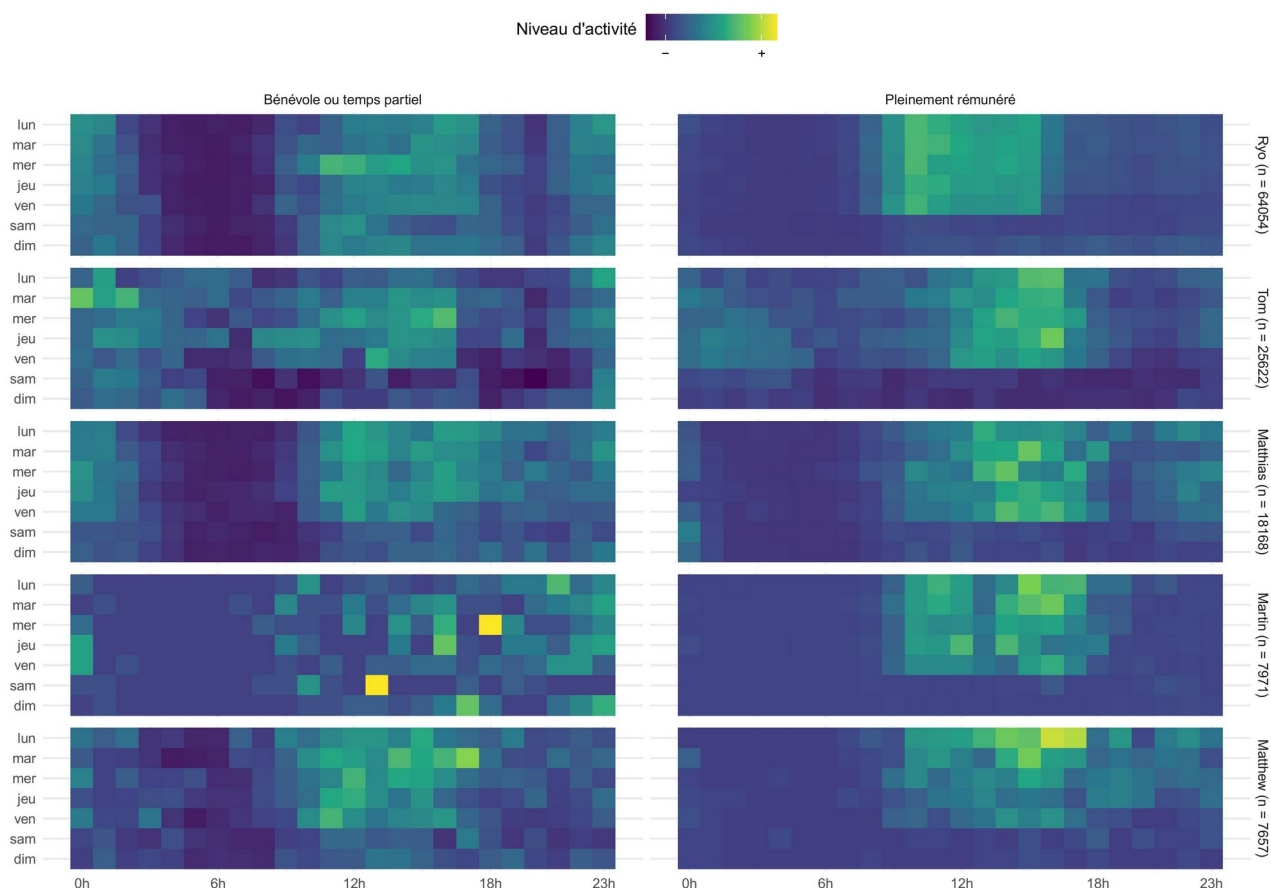


Figure 1. De contributeur bénévole à rémunéré : impact sur le rythme de travail

Ce graphique présente, pour cinq contributeurs (en ligne), deux cartes de fréquentation (heatmaps) qui décrivent leur niveau d'activité avant (colonne de gauche) et après (colonne de droite) qu'ils soient pleinement rémunérés pour leurs contributions. Le niveau d'activité est calculé à partir du nombre d'emails envoyés, transformé en une variable centrée réduite (moyenne nulle, variance et écart-type égaux à 1) ; il doit donc être interprété en termes relationnels, c'est-à-dire comme un moyen de comparer les niveaux d'activité, plutôt qu'en valeur absolue. **Champ** : 5 personnes et 123 472 emails, récoltés sur la période 2005–2020.

En résumé, contrairement à certaines idées reçues, le travail dans le logiciel libre n'est pas exclusivement gratuit. Dans le cas du projet Git, bien que la majorité des contributeurs restent bénévoles, les contributions rémunérées sont désormais la norme. Cette évolution est largement due aux grandes entreprises de la tech (Microsoft, Google, GitHub, etc.) qui embauchent les contributeurs bénévoles les plus actifs et les plus centraux dans le projet. Ces individus affichent ainsi des carrières de contributeur continues, qui passent du bénévolat au salariat.

La logique productive du travail gratuit

La présence de travail rémunéré au sein du projet Git constitue une première indication des liens entre informatique libre et marchande. Dans cette dernière partie, nous montrerons que ces liens ne passent pas uniquement par l'embauche et la rémunération. D'autres articulations moins visibles se jouent jusque dans l'obscurité du travail gratuit. Nous verrons que les contributions bénévoles peuvent trouver leur origine dans l'activité rémunérée, en explorant tout d'abord le contenu de ces contributions et leurs liens avec le *day job*, puis en montrant qu'elles s'effectuent régulièrement

pendant les heures de travail rémunéré. Enfin, nous discuterons de l'autonomie du travail des contributeurs à la lumière de ces nombreux liens entre informatique libre et marchande.

Des contributions bénévoles liées à l'activité rémunérée

L'écrasante majorité des contributeurs sont des professionnels de l'informatique, comme en témoigne le tableau 1. Les trois quarts des intitulés de poste témoignent d'une position dans l'industrie informatique, le plus souvent dans un métier de développement logiciel, principalement en tant que *Software Engineer* et *Developer*, tandis que 4 personnes seulement ont exercé un poste lié à la gestion de serveurs. Google est de loin le premier employeur de l'échantillon, avec 18 postes cumulés, suivi par GitHub (9 postes), Facebook (6 postes), GitLab et Microsoft (3 postes chacun). Parmi les autres employeurs se trouvent principalement d'autres géants du numérique (Apple, Amazon, Uber, IBM), des sociétés du secteur financier (Bloomberg, banques privées), des constructeurs de matériel informatique (Intel, Nvidia, Hewlett-Packard), des entreprises spécialisées dans le développement Linux et *open source* (RedHat, SUSE, Collabora, CodeWeavers) et bien d'autres encore, comme les studios d'animation de Walt Disney ou la plateforme de réservation d'hôtels en ligne Booking. Évidemment, toutes ces entreprises ne rémunèrent pas les contributions à Git, même si certaines sociétés comme Google encouragent et soutiennent la participation de leurs employés à des projets *open source*. La grande majorité de ces positions sont salariées, même si une dizaine de personnes au moins semblent avoir exercé en indépendant, parfois en complément d'une activité salariée.

Table 1. Les contributeurs de Git, un collectif de professionnels.

Intitulé du poste	Effectif (n)	%
Industrie informatique, dont...	146	77,2
<i>Engineer</i>	87	46,0
<i>Developer</i>	30	15,9
<i>Other (programmer, software architect, ...)</i>	18	9,5
<i>Executives (founder, CEO, CTO)</i>	7	3,7
<i>System Administrator</i>	4	2,1
Université, dont...	31	16,4
<i>Étudiants</i>	17	9,0
<i>Universitaires</i>	14	7,4
GSoC, Outreachy	12	6,4
Ensemble	189	100

Champ : intitulés de 189 postes renseignés auprès de 146 individus.

Quant au quart restant des positions occupées, 16,4 % sont issues du monde académique, dont plus de la moitié sont des étudiants en informatique (9 %). Les 7,4 % restants sont des professeurs d'Université, aujourd'hui titulaires pour la majorité d'entre eux. Enfin, 12 personnes ont bénéficié d'une bourse pour contribuer à Git, soit dans le cadre du *Google Summer of Code* (GSoC), qui

concerne principalement des étudiants, soit dans celui du programme de discrimination positive Outreachy, qui concerne plutôt des personnes ayant déjà commencé leurs carrières. Évidemment, une même personne n'occupe pas toujours une même position tout au long de sa carrière de contributeur. Un étudiant en informatique pourrait par exemple commencer à contribuer à l'Université, bénéficier quelques mois d'une bourse GSoC pour développer une fonctionnalité précise, et continuer à participer bénévolement au projet après avoir été embauché en tant qu'ingénieur à Google.

L'échantillon révèle également un très haut niveau de formation. Sur les 125 personnes dont le diplôme est connu (soit 60 % de l'échantillon), 43 % ont une licence et 37 % un master, la plupart du temps en informatique ou en mathématiques, tandis que 20 % (soit 25 personnes) ont un doctorat, dans des disciplines plus variées : informatique et mathématiques bien sûr, mais aussi physique et biologie. Par comparaison, l'enquête internationale StackOverflow de 2021 recense 50 % de licence, 25 % de master et 3 % de doctorat dans son échantillon de professionnels de l'informatique¹⁵. L'abondance de docteurs s'explique en partie par la présence d'universitaires parmi les contributeurs, mais seulement en partie, puisque la moitié d'entre eux quittent le monde académique après leur thèse pour prendre des postes dans l'informatique marchande.

Cette position professionnelle des membres du collectif entretient un lien étroit avec leurs contributions à Git. Comme le dit un célèbre adage de l'informatique libre, le travail des *hackers* est motivé par l'envie de « gratter là où ça démange »¹⁶, c'est-à-dire de résoudre les problèmes qu'ils rencontrent dans leur propre activité. Or une grande partie de l'activité des développeurs du libre a lieu dans le cadre de leur emploi, où ils sont régulièrement confrontés à des problèmes caractéristiques de l'informatique marchande. Ainsi de nombreuses contributions à Git sont-elles directement liées aux *day jobs* de leurs auteurs. Voici un exemple de contribution dont le message de *commit* fait explicitement référence à l'interaction de son autrice avec son supérieur hiérarchique :

Ajout du script git-stash

Lorsque mon patron a quelque chose à me montrer et que je dois mettre à jour [mon code], pour une raison ou une autre, je suis toujours en train de faire autre chose, et la commande git pull refuse de fonctionner dans ce cas.

J'ai écrit ce petit script pour sauvegarder les modifications que j'ai apportées, effectuer la mise à jour, puis revenir là où j'étais, mais après le commit mis à jour. [c]

Qu'ils soient rémunérés ou non pour leur travail, les contributeurs de Git sont donc des professionnels de l'informatique, hautement diplômés, qui occupent en général un emploi salarié dans l'informatique marchande, et sont confrontés à des problèmes d'écriture collective des codes.

Le temps du libre n'est pas le temps libre

¹⁵ « 2021 Developer Survey », StackOverflow, 2021, URL : <https://insights.stackoverflow.com/survey/2021#developer-profile-education>

¹⁶ En anglais, « *scratch their own itch* », selon une formule consacrée. Voir : Raymond E.S. (2001), *The Cathedral & the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary*, O'Reilly Media, Inc., 260 p.

Bien que le contenu des contributions bénévoles révèle les liens avec l'activité salariée, leur examen reste limité par la séparation que les contributeurs entretiennent entre l'informatique libre et marchande. Un deuxième faisceau d'indices confirme le rapport entre le travail gratuit sur Git et l'activité principale rémunérée : les horaires d'activité des contributeurs bénévoles, présentés dans la figure 4.

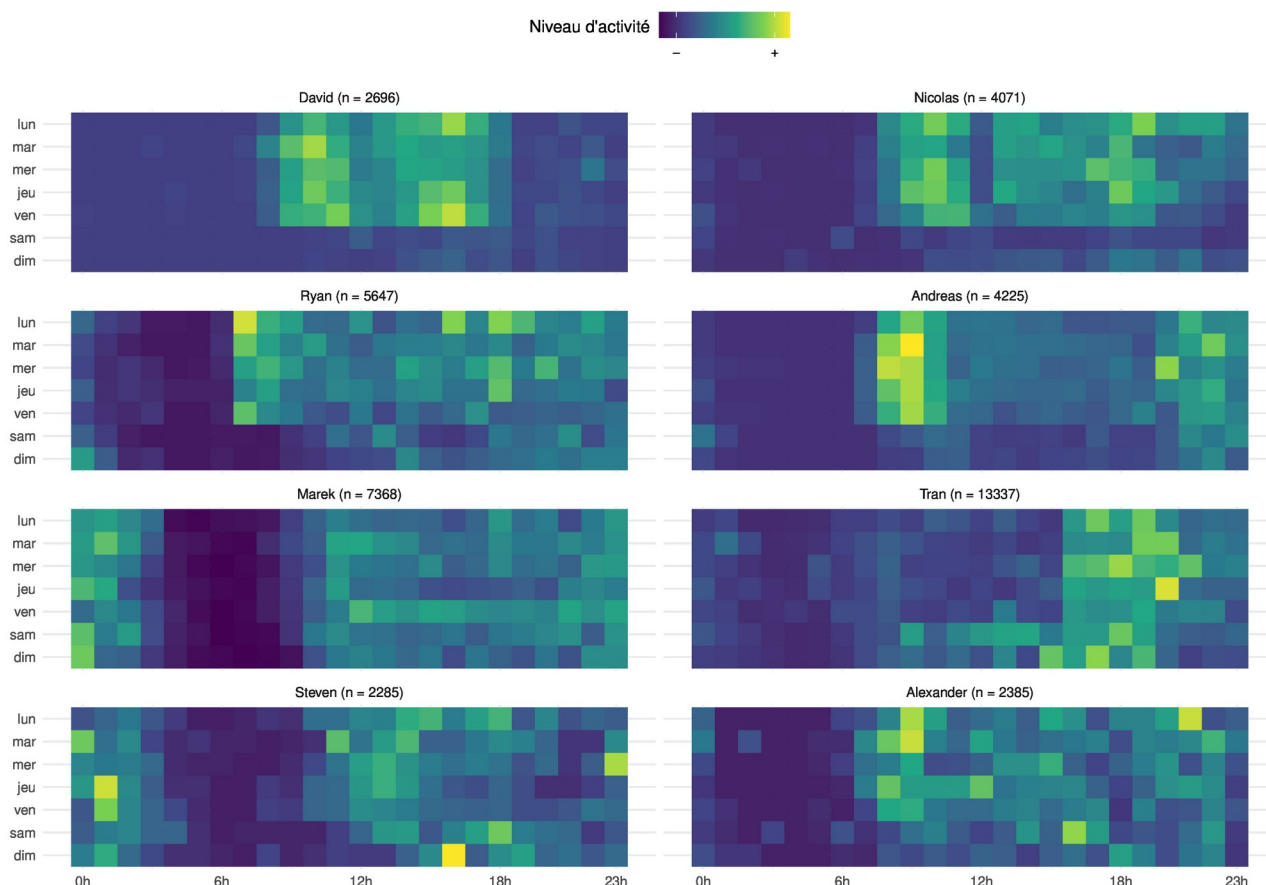


Figure 2. La diversité des horaires du travail gratuit

Ce graphique combine des cartes de fréquentation (heatmaps). Chacune représente, pour huit contributeurs différents, les moments de la semaine où il est le plus actif sur la liste de développement Git. Le niveau d'activité est calculé à partir du nombre d'emails envoyés, transformé en une variable centrée réduite (moyenne nulle, variance et écart-type égaux à 1); il doit donc être interprété en termes relationnels, c'est-à-dire comme un moyen de comparer les niveaux d'activité, plutôt qu'en valeur absolue. **Champ** : 8 personnes et 42 472 emails, récoltés sur la période 2005–2020.

Si la frontière entre informatique libre et marchande est effectivement étanche, nous pouvons alors formuler l'hypothèse que la très grande majorité des contributions s'effectuera en dehors des horaires de travail (*after hours*), par exemple la nuit ou le week-end. Cette distribution du travail gratuit se retrouve effectivement chez certains contributeurs. Certains sont plus actifs avant de partir au bureau vers 9h du matin (Andreas, Alexander, Ryan) ; d'autres quand ils en sortent vers la fin d'après-midi (Tran) ; bon nombre encore privilégient les heures sombres de la nuit, entre 22h et 4h du matin (Marek, Steven). Quant au week-end, il est vrai qu'un grand nombre de bénévoles (Ryan, Marek, Tran, Steven, Alexander) sont actifs pendant cette période, ce qui – rappelons-le – les distingue nettement des contributeurs salariés.

Or la question n'est pas tant de savoir si du travail gratuit a lieu hors des horaires d'emploi, mais plutôt : les bénévoles travaillent-ils à des moments que l'hypothèse des mondes antagonistes ne prédit et n'explique pas ? Quand nous analysons les horaires d'activité avec ce regard nouveau, ces données révèlent une dimension supplémentaire du travail gratuit. Revenons tout d'abord sur le week-end. Être inactif le samedi et le dimanche a beau être un facteur distinctif des contributeurs salariés, le week-end ne constitue pas pour autant un moment privilégié de l'activité bénévole. Pour certains, un jour de week-end est équivalent à un jour ouvré (Marek), à la différence près que les horaires de contribution y sont moins contraints (en week-end, Tran travaille plus en matinée et Ryan plutôt l'après-midi, contrairement à leur rythme du reste de la semaine). Mais beaucoup de bénévoles travaillent moins le samedi et le dimanche qu'en semaine (Andreas, Ryan, Alexander de la figure 4 ; Tom et Matthew de la figure 3). Quelques-uns, enfin, chôment le week-end (David, Nicolas) – au point que leurs rythmes de contribution ressemblent à ceux de leurs collègues salariés.

Ces constatations m'amènent aux horaires de contribution des jours ouvrés. Tout d'abord, même chez les bénévoles qui contribuent principalement tôt le matin ou tard le soir, une partie non négligeable de leur activité est clairement effectuée durant les heures de bureau. Rares sont les personnes qui, comme Tran, entretiennent une séparation stricte entre temps de travail gratuit et occupation salariée. Comment expliquer que le travail gratuit s'effectue, au moins en partie, sur le temps salarié alors que, par définition, ces bénévoles ne sont pas censés être payés pour leurs contributions à Git ?

La première explication est la pratique de la perruque, un phénomène bien connu de la sociologie du travail ouvrier (Anteby, 2003 ; De Certeau, 1990). Le terme, qui désigne habituellement le fait d'emprunter ou de voler du matériel appartenant à son employeur, prend une acception plus large dans le cas du logiciel libre, recouvrant à la fois l'usage des ressources de son entreprise (ordinateurs bien sûr, mais parfois aussi serveurs internes) ainsi que du temps de travail rémunéré. La pratique de la perruque pour développer des logiciels libres est une pratique répandue (Vicente, 2015) et avérée depuis les années 1980 (Alcaras, 2020). Dans ce cas, les contributions à Git se font au nez et à la barbe de la hiérarchie, dans le secret de son *cubicle* par exemple.

Contribuer depuis son lieu de travail s'explique, deuxièmement, par le fait que des ingénieurs utilisent Git dans le cadre de leur activité salariée. Ils rencontrent alors des problèmes spécifiques soit à l'organisation du travail de leur entreprise, soit aux technologies souvent propriétaires de leur infrastructure interne, par exemple avec des serveurs à haute fréquence ou les outils logiciels d'un studio d'animation. Pour régler ces problèmes, ces contributeurs vont adapter Git à leur usage professionnel et contribuer cette modification du code au projet officiel, une pratique courante qualifiée d'*upstreaming*, littéralement « faire remonter en amont ». Développer ces solutions nécessite donc d'être présent sur le lieu de travail, puisqu'elles découlent de problèmes que les ingénieurs ne rencontrent que dans ce contexte.

Enfin, une dernière variable joue sur la proportion de contributions effectuées au bureau et pendant les horaires de travail rémunéré : le degré d'autonomie temporelle (Ganault, 2022) dont les contributeurs jouissent dans leur *day job*. Certains environnements professionnels sont plus rigides que d'autres, notamment quand des algorithmes sensibles ou des clients du complexe militaro-industriel imposent des protocoles de sécurité contraignants. Les contributeurs peuvent alors rencontrer des difficultés techniques, voire juridiques, ne serait-ce que pour interagir avec la liste de diffusion depuis leur lieu de travail. À l'inverse, d'autres contextes professionnels laissent une très

grande liberté aux contributeurs. Le cas des universitaires (David et Nicolas) est ici exemplaire : leurs contributions bénévoles se font presque exclusivement du lundi au vendredi, entre 10h et 18h – laissant même une pause rituelle pour le déjeuner dans le cas d'un chercheur français.

Le travail gratuit épouse ainsi, parfois très étroitement, le temps du travail salarié. Cette synchronicité des rythmes de l'activité gratuite et rémunérée corroborent l'hypothèse de leur continuité plutôt que de leur séparation. Voilà pourquoi, lors des moments d'observation, la distinction entre contributeurs bénévoles et rémunérés paraît floue, tant leurs pratiques sont similaires. Même un ingénieur de Google, contribuant régulièrement à Git pendant ses horaires de bureau et chargé du développement d'une infrastructure interne construite autour de Git, peut se considérer comme un contributeur bénévole : « moi, je *hacke* sur Git parce que ça m'amuse (ça me change du *hacking* morne que je fais pour manger) et parce que j'aime l'utiliser pour m'aider dans mon travail morne de *hacking*. Mais je ne reçois certainement pas d'argent quand je *hacke* sur Git » [14]. Le temps libre n'est donc pas le temps du libre, mais seulement un de ses moments.

Le peintre et la cour

En 2015, lors d'une rare discussion au sujet des conditions de travail des contributeurs, les esprits s'échauffent et Junio, le mainteneur du projet, expose sa vision en ces termes :

Personnellement, je pense que les bons développeurs *open source* devraient être comme des peintres de cour. Ils ont peut-être à peindre les enfants de leur employeur pour gagner leur dû, même s'ils n'aiment pas le faire, mais leur production aura au final enrichi le grand public. Il est possible que leurs employeurs n'aient été que des sangsues qui exploitent leurs serfs, exactement comme les entreprises malveillantes des temps modernes, et je comprends ceux qui s'opposent à ma vision simpliste du monde. [15]

Quand l'employé de Google écrit ses lignes, Hamano a la chance de vivre pleinement de son art grâce à la générosité du mécène californien – après avoir été longtemps ce peintre de cour, contraint de gagner sa croûte en écrivant du code sans saveur pour ses patrons, trouvant quand il le peut le temps d'exercer librement son activité pour créer selon ses propres envies et ses propres exigences. Une frontière familière se dessine : d'un côté, l'informatique marchande, qui paie mais ne satisfait pas ; de l'autre, l'informatique libre, qui importe mais ne rémunère pas. Certes, l'analogie est un peu brute et se réfère davantage à un imaginaire commun qu'aux véritables conditions de travail et aux identités professionnelles des peintres du Moyen-Âge ou de la Renaissance (Moulin, 1983, p. 198). Mais cet imaginaire en dit long sur celui de l'informatique libre : les contributions sont bénévoles, effectuées sur le temps libre, sans rapport avec l'activité rémunérée.

Or considérer que le rapport à l'informatique marchande passe uniquement par la rémunération, c'est effacer toute une partie des liens et des échanges qui se nouent au cœur du travail gratuit. Dans une analyse remarquable qui fait dialoguer sociologie du travail et enquêtes sur le bénévolat, Maud Simonet montre que le travail gratuit est une véritable « carrière parallèle » (Simonet, 2010, p. 36), articulée aux mondes du travail, et qu'elle fait l'objet d'investissements différenciés. Pour certains, cette carrière parallèle permet, à l'image des ingénieurs de cour que décrit Junio, « de prolonger et de compléter [...] l'activité exercée professionnellement, quand il ne compense pas tout simplement une activité ou une carrière professionnelle ressentie comme trop peu engageante » (*ibid.*, p. 44). Pour d'autres, « investis bénévolement et professionnellement dans le même monde du travail, l'activité bénévole est vécue comme une continuité, et même une dimension de leur activité

professionnelle. Ils présentent alors leur travail comme une totalité » (*ibid.*, p. 47). Cet engagement subjectif et objectif dans le travail gratuit explique qu'un contributeur puisse percevoir son statut de bénévole, puis de salarié, comme des périodes successives d'une seule et même carrière continue.

Considérons donc les carrières des 25 principaux contributeurs entre 2005 et 2020. Une moitié d'entre eux conservent leur statut tout au long de leur carrière : quatre ont toujours été rémunérés (dont trois dès la conception de Git), tandis que les huit autres sont restés bénévoles. L'autre moitié des gros contributeurs voient leur statut évoluer, surtout à partir de 2010. La majeure partie d'entre eux contribuent bénévolement durant de nombreuses années, parfois plus de dix ans, avant d'être embauchés par de grandes entreprises ou, pour une minorité, en se mettant à leur compte en tant qu'indépendants. Git représente ainsi un travail constant au cours de leurs carrières professionnelles ; en revanche, la place de ce travail dans leurs carrières change dès lors qu'elle devient leur activité rémunérée. Pour le contributeur, sa carrière jusqu'alors invisible est reconnue par les entreprises. L'embauche n'a donc rien d'une professionnalisation et tout d'une institutionnalisation des carrières de contribution dans l'informatique marchande.

L'analogie du peintre et de la cour souligne enfin, à juste titre, la dépendance des contributeurs vis-à-vis de l'informatique marchande – les bénévoles dépendent de leurs *day jobs*, les salariés de leurs employeurs, les indépendants de leurs clients. Cette dépendance économique s'explique notamment par la vision des mondes antagonistes : la majorité des contributeurs investissent le logiciel libre pour se consacrer aux questions de production technique, et n'envisagent pas une seule seconde de s'organiser afin de créer des collectifs économiquement indépendants de l'informatique marchande. Pour des contributeurs qui rechignent devant le sale boulot (Hughes, 1984, pp. 338-347) des tâches administratives et marchandes (Jourdain, 2014), la dépendance économique est le prix à payer pour se consacrer à son vrai travail (Bidet, 2011).

Malgré sa dépendance économique et ses multiples liens avec l'informatique marchande, l'informatique libre conserve une certaine autonomie, sans être totalement contrôlée par les grandes entreprises de la tech. En effet, l'activité des contributeurs, rémunérés ou non, est loin d'être entièrement dédiée à des tâches qui profitent de manière évidente et directe à leurs employeurs. De même, leur manière de contribuer échappe à certaines formes de rationalisation du travail informatique qui sont courantes au sein du monde marchand. Les contributeurs de Git rapportent par exemple que le rythme lent du développement libre – fondé sur le consensus, le soin des choses (Denis & Pontille, 2022) et un horizon de long terme – entre régulièrement en contradiction avec le rythme rapide du développement marchand – fondé sur la subordination, le prototypage et un horizon de court terme. Pour reprendre la métaphore de Junio, les peintres prennent leur temps pendant que la cour trépigne. La contribution à l'informatique libre se distingue ainsi d'autres formes de travail gratuit, dans la mesure où une fraction masculine experte de la profession conserve un certain contrôle sur le contenu et le rythme de sa production.

Conclusion

Cette étude sur la production de Git met en évidence la continuité entre travail gratuit et travail rémunéré dans l'industrie informatique. Nous avons tout d'abord présenté la puissance de la vision des « mondes antagonistes » : cette dernière efface les statuts des contributeurs des traces du travail informatique, ce qui renforce en retour l'impression que le libre existe en dehors de l'informatique marchande. Nous avons alors proposé d'enrichir ces données pour rendre compte des différents

statuts des contributeurs. Nous avons ensuite montré que, contrairement aux idées reçues, tout le travail fait dans le logiciel libre ne relève pas du travail gratuit. Dans le cas de Git, le travail rémunéré est désormais la règle en matière de nombre de contributions, même s'il reste l'exception en termes de nombre de contributeurs – une dynamique courante dans l'économie numérique, où seule une petite partie de travailleurs parvient à vivre de son activité en ligne. Sa présence augmente constamment depuis la création du projet, en grande partie grâce à l'embauche de bénévoles par des grandes entreprises de la tech (comme Microsoft, Google, ou GitHub). Enfin, nous avons observé comment le travail gratuit au sein de Git s'articule à l'activité rémunérée à l'extérieur du projet, soulignant que même les contributions bénévoles sont souvent liées à des préoccupations professionnelles et s'effectuent parfois durant les heures de travail salarié.

Les liens étroits et les échanges constants entre l'informatique libre et marchande rendent donc la vision des « mondes antagonistes » caduque, tandis que l'hypothèse continuiste permet à la fois de mieux décrire et de mieux comprendre les dynamiques du travail dans son ensemble, qu'il soit gratuit et rémunéré. Cette constatation a deux grandes implications : l'une qui touche aux communs numériques, et l'autre qui concerne les études sur le travail gratuit en général.

La première implication de cette étude concerne le devenir des communs numériques. L'informatique libre est souvent perçue comme un "collectif indépendant d'autres formes de pouvoir constitué" (Kelty, 2008, p. 3) et parfois caractérisée comme une alternative concrète au capitalisme (Gorz, 2003). Le travail gratuit a ainsi été vu comme un moyen de créer une forme d'autonomie face aux logiques marchandes. Dans cette perspective, toute irruption de l'informatique marchande dans l'informatique libre est interprétée comme une menace pour l'autonomie des communs numériques. Quand des géants du numérique comme Microsoft investissent massivement dans l'open source, notamment en embauchant des contributeurs bénévoles, le déclin du travail gratuit au profit du travail rémunéré est perçu comme un sinistre augure, annonçant la transformation du libre en "communs du capital". Cette ligne critique, qui présuppose l'hypothèse des mondes antagonistes, contient une tension paradoxale : le travail gratuit est à la fois une forme d'exploitation et une condition de l'autonomie du logiciel libre, ce qui rend son remplacement par du travail rémunéré inacceptable.

L'hypothèse continuiste offre une perspective différente. Elle rappelle tout d'abord que, dans de nombreux cas, les contributeurs ne s'opposent pas à la présence de grandes entreprises dans le logiciel libre. Leur objectif n'est pas tant l'indépendance que de retrouver suffisamment de contrôle pour accomplir leur travail correctement, selon leurs propres critères — il s'agit donc d'abord d'une logique productive avant d'être émancipatrice. Au contraire, pour de nombreux contributeurs bénévoles, l'embauche par de grandes entreprises signifie être payé pour faire un travail que l'on aime et que l'on estime, le « vrai travail », tout en marquant une forme de reconnaissance symbolique pour tout le travail gratuit qu'ils ont déjà accompli. L'hypothèse continuiste montre également que, malgré ces échanges permanents avec l'informatique marchande, les contributeurs conservent une forme de contrôle sur leur travail. Plutôt que de parler de dépendance ou d'indépendance, il serait plus pertinent d'analyser la position des logiciels libres à travers le concept d'autonomie relative (Sapiro, 2019), qui invite à réfléchir en termes de degrés — jusqu'à quel point un logiciel libre est-il autonome ? — et en termes relationnels — avec quelles entités cette autonomie se négocie-t-elle ?

Cette perspective est aussi une invitation à préciser les différentes configurations des logiciels libres vis-à-vis de l'informatique marchande. En effet, l'hypothèse continuiste offre un cadre analytique à

l'intérieur duquel plusieurs formes d'articulations, d'échanges et de relations entre l'informatique libre et marchande sont envisageables. Penser la continuité du travail gratuit et rémunéré ne signifie pas que toutes les continuités sont identiques, bien au contraire. Or l'exploration de ces différentes configurations dépend du choix et de la constitution des études de cas. Cet article a mis en évidence le cas de Git parce qu'il est typique de ces logiciels libres, jusqu'ici peu étudiés par les sciences sociales, qui sont devenus des ressources stratégiques pour l'informatique marchande¹⁷. Ces projets, aussi emblématiques soient-ils de cette tendance récente d'investissements massifs et explicites, occupent toutefois des positions spécifiques. Ils se distinguent ainsi des collectifs plus politiques étudiés par les travaux pionniers de l'informatique libre¹⁸ ou encore des très nombreux logiciels maintenus bénévolement par une personne isolée, et qui sont pourtant des parties essentielles de nos infrastructures numériques¹⁹. L'analyse de ces différents cas pourrait enrichir notre compréhension des articulations entre travail gratuit et rémunéré tout en ouvrant de nouvelles perspectives sur le devenir différencié des communs numériques.

Cette étude confirme enfin l'intérêt de considérer les activités gratuites et bénévoles comme du travail (Simonet, 2004), concept qui permet de suivre une même activité sous toutes ses formes. Dans nombre de cas, distinguer travail gratuit et rémunéré est, de fait, presque impossible. La difficulté de catégoriser certaines contributions nous éclaire sur l'absurdité de ce que Bernard Friot nomme la rémunération capitaliste (Friot, 2021), c'est-à-dire un paiement à la tâche. Du point de vue des ingénieurs, leurs contributions s'intègrent à un tout plus vaste, celui de leur activité quotidienne d'écriture des codes. Or certaines des tâches qu'ils doivent mener à bien pour accomplir ce travail les amènent à régler des problèmes pour lesquels ils ne peuvent pas être rémunérés car ces tâches, pourtant nécessaires, ne contribuent pas directement aux profits de l'employeur. Puisque le processus de production informatique ne peut pas être entièrement pris en charge par les institutions marchandes, la part négligée par ces dernières s'exprime dans l'informatique libre, à travers une autre forme d'institution économique que nous appelons les biens industriels publics (Alcaras, 2020). Voilà pourquoi une même logique productive se trouve fracturée entre travail gratuit et rémunération capitaliste, qui s'entremêlent et se relaient tout au long de l'écriture informatique — de jour comme de nuit.

¹⁷ Ces projets ont souvent pour objet des problématiques infrastructurelles critiques pour toute l'industrie : la gestion de versions (Git), les systèmes d'exploitation (Linux et certaines de ces déclinaisons, comme RedHat et Ubuntu), l'administration de serveurs et du cloud en général (Docker, Kubernetes), la gestion de données massives (Apache Spark, Elasticsearch), la sciences des données (Rstudio, Jupyter) ou encore l'apprentissage automatique (Scikit-learn, TensorFlow, PyTorch).

¹⁸ Nous pensons particulièrement au système d'exploitation Debian (Auray, 2003; Coleman, 2013) ou, dans le cas français, au logiciel Spip (Demazière *et al.*, 2007).

¹⁹ Ces logiciels font souvent irruption dans l'actualité généraliste lorsqu'ils sont à l'origine de failles de sécurité majeures, comme la faille Heartbleed d'OpenSSL découverte en 2014, ou la porte dérobée dans le logiciel xz révélée en mars 2024.

Bibliographie

Abbott A. (1988), *The system of professions. An essay on the division of expert labor*, Chicago, University of Chicago Press.

Abdelnour S. & Bernard S. (2018), « Vers un capitalisme de plateforme ? Mobiliser le travail, contourner les régulations », *La Nouvelle Revue du Travail*, n°13.

Akrich M. (2012), *Les outils de discussion comme communautés en ligne : outils de description et méthodes d'analyse*, Paris, Centre de Sociologie de l'Innovation.

Alcaras G. (2020), « Des biens industriels publics. Genèse de l'insertion des logiciels libres dans la Silicon Valley », *Sociologie du travail*, vol. 62, n°3.

Alcaras G. & Larribeau A. (2022), « Codes. L'informatique comme elle s'écrit », *RESET*, n°11, .

Alexandre O. (2023), *La Tech: Quand la Silicon Valley refait le monde*, Paris XIXe, SEUIL.

Anteby M. (2003), « La « perruque » en usine: approche d'une pratique marginale, illégale et fuyante », *Sociologie du travail*, vol. 45, n°4, p. 453-471.

Auray N. (2003), « La régulation de la connaissance : arbitrage sur la taille et gestion aux frontières dans la communauté Debian », *Revue d'économie politique*, vol. 113, n°[Suppl.], p. 161-182.

Beaudouin V. (2021), « Chapitre 8. Collectifs d'amateurs en ligne : produire de la connaissance et du lien social », in Dagiral, Éric et Martin, Olivier (dir.), *Les liens sociaux numériques*, Paris, Armand Colin, p. 179-197.

Bergström M. (2016), « (Se) correspondre en ligne », *Sociétés contemporaines*, vol. 104, n°4, p. 13-40.

Bialski P. (2024), *Middle Tech: Software Work and the Culture of Good Enough*, Princeton, Princeton University Press.

Bidet A. (2011), *L'engagement dans le travail. Qu'est-ce que le vrai boulot ?*, Paris, Presses universitaires de France « Le lien social ».

Blanchard S., Boni-Le Goff I., Delage P. & Puech I. (2021), « Introduction. (Re)configurations du travail domestique », *Travail, genre et sociétés*, vol. 46, n°2, p. 27-31.

Bowker G. C., Baker K., Millerand F. & Ribes D. (2010), « Toward Information Infrastructure Studies: Ways of Knowing in a Networked Environment », in Hunsinger J., Klastrup L. et Allen M. (dir.), *International Handbook of Internet Research*, Dordrecht, Springer Netherlands, p. 97-117.

Broca S. (2013), *Utopie du logiciel libre. Du bricolage informatique à la réinvention sociale*, Paris, Le Passager Clandestin.

- Broca S.** (2017), « Le digital labour, extension infinie ou fin du travail ? », *Tracés. Revue de Sciences humaines*, n°32, p. 133-144.
- Burrell J. & Fourcade M.** (2021), « The Society of Algorithms », *Annual Review of Sociology*, vol. 47, n°Volume 47, 2021, p. 213-237.
- Chadeau A. & Fouquet A.** (1981), « Peut-on mesurer le travail domestique ? », *Économie et statistique*, vol. 136, n°1, p. 29-42.
- Coleman G.** (2013), *Coding freedom. The ethics and aesthetics of hacking*, Princeton, Princeton University Press.
- Couture S.** (2012), « L'écriture collective du code source informatique. Le cas du commit comme acte d'écriture », *Revue d'anthropologie des connaissances*, vol. 6, n°1, p. 21-42.
- De Certeau M.** (1990), *L'invention du quotidien*, Paris, Gallimard.
- Demazière D., Horn F. & Zune M.** (2007), « Des relations de travail sans règles ? L'énigme de la production des logiciels libres », *Sociétés contemporaines*, vol. 2, n°66, p. 101-125.
- Denis J. & Pontille D.** (2022), *Le soin des choses*, Paris, La Découverte.
- Depecker T., Déplaud M.-O. & Larchet N.** (2018), « La philanthropie comme investissement. Contribution à l'étude des stratégies de reproduction et de légitimation des élites économiques », *Politix*, vol. 121, n°1, p. 9-27.
- Derrien M., Rossigneux-Méheust M. & Sarzier M.** (2023), « Repousser les frontières du travail: Propositions pour une autre approche de la vieillesse », *Sociétés contemporaines*, vol. 129, n°1, p. 5-21.
- Desrosières A.** (2001), « Entre réalisme métrologique et conventions d'équivalence : les ambiguïtés de la sociologie quantitative », *Genèses*, vol. 43, n°2, p. 112-127.
- Desrosières A.** (2008), *Pour une sociologie historique de la quantification. L'Argument statistique*, Paris, Presses des Mines.
- Didier E.** (2013), *La représentativité en statistique*, Paris, INED Éditions.
- Ensmenger N.** (2010), *The computer boys take over. Computers, programmers, and the politics of technical expertise*, Cambridge, MIT Press.
- Friot B.** (2021), « Le salaire et la propriété de l'outil, droits économiques de la personne », *L'Homme et la Société*, vol. 212, n°1, p. 243-273.
- Ganault J.** (2022), « Autonomie temporelle dans le travail salarié et articulation des temps sociaux. Qui peut organiser son temps quotidien ? », *Sociologie*, vol. 13, n°4, p. 399-419.
- Gorz A.** (2003), *L'immatériel : connaissance, valeur et capital*, Paris, Galilée « Débats ».
- Hicks M.** (2017), *Programmed inequality: how Britain discarded women technologists and lost its edge in computing*, Cambridge, MA, MIT Press « History of computing ».
- Himanen P.** (2001), *The hacker ethic, and the spirit of the information age*, New York, Random House.

- Hughes E. C.** (1984), *The Sociological Eye: Selected Papers*, London New York, Routledge.
- Jourdain A.** (2014), « L'origine sociale de la vigilance entrepreneuriale. Quand les artisans d'art se font entrepreneurs », *Sociologie du travail*, vol. 56, n°3, p. 279-297.
- Kelty C. M.** (2008), *Two bits: The cultural significance of free software*, Durham, Duke University Press.
- Litman L., Robinson J., Rosen Z., Rosenzweig C., Waxman J. & Bates L. M.** (2020), « The persistence of pay inequality: The gender pay gap in an anonymous online labor market », *PLOS ONE*, vol. 15, n°2, p. e0229383.
- Moulin R.** (1983), « De l'artisan au professionnel : l'artiste », *Sociologie du travail*, vol. 25, n°4, p. 388-403.
- Neely M. T., Sheehan P. & Williams C. L.** (2023), « Social Inequality in High Tech: How Gender, Race, and Ethnicity Structure the World's Most Powerful Industry », *Annual Review of Sociology*, vol. 49, n° 49, 2023, p. 319-338.
- Nielsen J.** (1998), « Community is Dead; Long Live Mega-Collaboration ». URL : <https://web.archive.org/web/19980128113201/http://www.useit.com/alertbox/9708b.html> (page consultée le 26/03/24)
- Ollion É.** (2016), « L'abondance et ses revers. Big data, open data et recherches sur les questions sociales », *Informations sociales*, n°191, p. 70-79.
- O'Neil M., Cai X., Muselli L. & Zacchiroli S.** (2022), « Co-producing industrial public goods on GitHub: Selective firm cooperation, volunteer-employee labour and participation inequality », *New Media and Society*, vol. 5, n°26, p. 2556-2592.
- Pontille D.** (2002), « La signature scientifique. Authentification et valeur marchande », *Actes de la recherche en sciences sociales*, vol. 141-142, n°1, p. 72-78.
- Pudal P.** (2016), *Retour de flammes*, Paris, La Découverte.
- Rikap C. & Lundvall B.-Å.** (2020), « Big tech, knowledge predation and the implications for development », *Innovation and Development*, vol. 3, n°12, p. 389-416.
- Sapiro G.** (2019), « Repenser le concept d'autonomie pour la sociologie des biens symboliques », *Biens symboliques*, n°4.
- Simonet M.** (2002), « "Give back to the community" : le monde du bénévolat américain et l'éthique de la responsabilité communautaire », *Revue française des affaires sociales*, n°4, p. 167-188.
- Simonet M.** (2004), « Penser le bénévolat comme travail pour repenser la sociologie du travail », *Revue de l'IRES*, vol. 1, n°44, p. 141-155.
- Simonet M.** (2010), *Le travail bénévole. Engagement citoyen ou travail gratuit ?*, Paris, La Dispute.
- Simonet M.** (2018), *Travail gratuit : la nouvelle exploitation ?*, Textuel.
- Simonet M.** (2020), « Le workfare ou la mise au travail gratuit des allocataires de l'aide sociale », *Regards croisés sur l'économie*, vol. 27, n°2, p. 184-193.

- Star L.** (1999), « The Ethnography of Infrastructure », *American Behavioral Scientist*, vol. 43, n°3, p. 377-391.
- Stevens H.** (2016), « Mais où sont les informaticiennes ? », *Travail, genre et sociétés*, vol. 36, n°2, p. 167-173.
- Terrell J., Kofink A., Middleton J., Raine C., Murphy-Hill E., Parnin C. & Stallings J.** (2017), « Gender differences and bias in open source: pull request acceptance of women versus men », *PeerJ Computer Science*.
- Tsay J., Dabbish L. & Herbsleb J.** (2014), « Influence of social and technical factors for evaluating contribution in GitHub », in *Proceedings of the 36th international conference on Software engineering*, ACM, p. 356-366.
- Turner F.** (2008), *From Counterculture to Cyberculture. Stewart Brand, The Whole Earth Network, and the Rise of Digital Utopianism*, Chicago, University of Chicago Press.
- Vicente M.** (2015), « Les parcours des développeurs de logiciels libres : vecteur de diffusion d'une innovation sociale », *Sociologies pratiques*, vol. 2, n°31, p. 55-64.
- Vinck D.** (1999), « Les objets intermédiaires dans les réseaux de coopération scientifique. Contribution à la prise en compte des objets dans les dynamiques sociales », *Revue Française de Sociologie*, vol. 40, n°2, p. 385-414.
- Weber F.** (1986), « Le travail hors de l'usine. Bricolage et double activité », *Revue d'Études en Agriculture et Environnement*, vol. 3, n°1, p. 13-36.
- Weber F.** (1989), *Le travail à-côté : étude d'ethnographie ouvrière*, Editions de l'Ecole des Hautes Etudes en Sciences Sociales.
- Zelizer V.** (2001), « Transactions intimes », *Genèses*, vol. 42, n°1, p. 121-144.

Annexe électronique

Cette annexe a pour but de fournir des informations plus détaillées sur les sources sur lesquelles s'appuie l'article.

Emails

Messages de la liste de discussion du projet Git, hébergés sur l'archive web public-inbox.org. Par ordre de citation :

[1] 30 avril 2018, brian m. carlson, « Re: Bug: format-patch MIME boundary not added to cover letter when attach enabled », URL :

<https://public-inbox.org/git/20180430115909.GB13217@genre.crustytoothpaste.net/>.

[2] 19 mars 2021, brian m. carlson, « Re: Are there any publicly known funders (companies or otherwise) of git development? », URL :

<https://public-inbox.org/git/YFURduetSoPW%2FScd@camp.crustytoothpaste.net/>.

[3] 20 août 2018, Christian Couder, « Re: Git project leadership committee », URL : <https://public-inbox.org/git/CAP8UFD3agHbDuDts7SsOaLx5A6vZ=HUxkCWYTwTSw1x5Zvbyw@mail.gmail.com/>.

[4] 19 août 2018, Ævar Arnfjörð Bjarmason, « Re: Git project leadership committee », URL :

<https://public-inbox.org/git/87zhxipmv9.fsf@evledraar.gmail.com/>.

[5] 8 septembre 2011, Jeff King, « Re: The importance of including http credential caching in 1.7.7 », URL : <https://public-inbox.org/git/20110908191053.GA16064@sigill.intra.peff.net/>.

[6] 15 mars 2020, Jakub Narebski, « Re: [RFC] possible idea for GSoC 2020 », URL :

<https://public-inbox.org/git/86zhch7kyz.fsf@gmail.com/>.

[7] 26 avril 2014, David Kastrup, « [PATCH 2/2] Mention « git blame » improvements in release notes », URL : <https://public-inbox.org/git/1398470210-28746-2-git-send-email-dak@gnu.org/>.

[8] 26 avril 2014, Junio C Hamano, « Re: [PATCH 2/2] Mention « git blame » improvements in release notes », URL : <https://public-inbox.org/git/7vmwf8huey.fsf@alter.siamese.dyndns.org/>.

[9] 28 avril 2014, David Kastrup, « Re: [PATCH 2/2] Mention « git blame » improvements in release notes », URL : <https://public-inbox.org/git/87ha5d4480.fsf@fencepost.gnu.org/>.

[10] 26 juillet 2005, Junio C Hamano, « A note from the new GIT maintainer », URL :

<https://public-inbox.org/git/7vmzo8ss2l.fsf@assigned-by-dhcp.cox.net/>.

[11] 25 mai 2010, Junio C Hamano, « What's cooking in git.git (May 2010, #05; Tue, 25) », URL :

<https://public-inbox.org/git/7vpr0js197.fsf@alter.siamese.dyndns.org/>.

[12] 2 mars 2011, Jeff King, « Re: [RFC] git blame-tree », URL :

<https://public-inbox.org/git/20110302174056.GB18957@sigill.intra.peff.net/>.

[13] 3 septembre 2015, Johannes Schindelin, « On a personal note », URL : <https://public-inbox.org/git/alpine.DEB.1.00.1509031156510.29350@s15462909.onlinehome-server.info/>.

[14] 28 février 2007, "Shawn O. Pearce", « Re: Google summer of code 2007 », URL :

<https://public-inbox.org/git/20070228154701.GB5479@spearce.org/>.

[15] 10 mars 2015, Junio C Hamano, « Re: Promoting git developers », URL : <https://public-inbox.org/git/xmqqzj7l1eje.fsf@gitster.dls.corp.google.com/>.

Commits

Commits (contributions de code) du dépôt git cités dans l'article. Par ordre de citation :

[a] 26 mars 2009, Michael Hendricks, « format-patch: add arbitrary email headers », commit d7d9c2d049, URL :

<https://github.com/git/git/commit/d7d9c2d04962b5aec34ec891d82dd74262306c56>.

[b] 24 mars 2017, Lars Schneider, « travis-ci: build and test Git on Windows », commit 029aeed55, URL : <https://github.com/git/git/commit/029aeed55f6bfe8014e8ffe5fc7a6f2e5b110fc>.

[c] 30 juin 2007, Nanako Shiraishi, « Add git-stash script », commit f2c66ed196, URL : <https://github.com/git/git/commit/f2c66ed196d1d1410d014e4ee3e2b585936101f5>.

Observations

Tableau 1. Liste des observations menées en conférence open source

Date	Événements	Taille	Organisation	Lieu
2017, 28 mars	Git Meetup	30	ToucanToco	Paris, FR
2017, 20 avr.	Git Meetup	30	GRDF	Paris, FR
2017, 02 mai	Git Meetup	30	VentePrivée	Paris, FR
2017, 21 sept.	Git Meetup	200	Société Générale	Paris, FR
2017, 11–12 nov.	Git Sprint	30	Bloomberg	Londres, GB
2018, 07–08 mars	Git Merge (\$)	350	GitHub	Barcelone, ES
2018, 15–17 oct.	GitHub Universe (\$)	1800	GitHub	San Francisco, USA
2018, 15–16 nov.	Gerrit User Summit	50	Cloudera	Palo Alto, USA
2019, 31 jan.–01 fév.	Git Merge (\$)	350	GitHub	Bruxelles, BE
2019, 01–02 fév.	FOSDEM	8000+	FOSDEM	Bruxelles, BE
2019, 10–11 déc.	Paris Open Source Summit	400	Weyou Group	Paris, FR

Remarques : les conférences signalées (\$) sont payantes. La colonne taille se rapporte au nombre estimé de participant·es.

Webographie

Toutes les sources web ont été consultées et vérifiées en octobre 2024.

15 août 1997, « Community is Dead; Long Live Mega-Collaboration », Jakob Nielsen, URL : <https://web.archive.org/web/19980128113201/http://www.useit.com/alertbox/9708b.html>

12 avril 2018, « Open source is everywhere: survey results part 1 », Keenan Szulik, URL : <https://blog.tidelift.com/open-source-is-everywhere-survey-results-part-1>

24 juin 2018, « Les Communs numériques sont-ils condamnés à devenir des “Communs du capital” ? », calimaq, URL : <https://scinfolex.com/2018/06/24/les-communs-numeriques-sont-ils-condamnes-a-devenir-des-communs-du-capital/>

2021, « 2021 Developer Survey », StackOverflow, URL : <https://insights.stackoverflow.com/survey/2021#developer-profile-education>

2022, « 2022 Developer Survey », StackOverflow, URL : <https://survey.stackoverflow.co/2022/#section-version-control-version-control-systems>

4 janvier 2022, « Creating », Google Open Source, URL : https://opensource.google/documentation/reference/creating#personal_projects

15 septembre 2022, « Open source patching », Google Open Source, URL : <https://opensource.google/documentation/reference/patching>

Janvier 2022, « Le pillage de la communauté des logiciels libres », Mathieu O’Neil, Laure Muselli, Fred Pailler & Stefano Zacchiroli, *Le Monde diplomatique*, URL : https://www.monde-diplomatique.fr/2022/01/O_NEIL/64221

Mai 2023, « May 2023 National Occupational Employment and Wage Estimates », *U.S. Bureau of Labor Statistics*, URL : https://www.bls.gov/oes/2023/may/oes_nat.htm#00-0000

2024, « Salaire moyen d'un Ingenieur Informatique en 2024 - France », *Talent.com*, URL : [https://fr.talent.com/salary?](https://fr.talent.com/salary?job=ingenieur+informatique#:~:text=Le%20salaire%20m%C3%A9dian%20pour%20les,%E2%82%AC%2050%20000%20par%20an)

[job=ingenieur+informatique#:~:text=Le%20salaire%20m%C3%A9dian%20pour%20les,%E2%82%AC%2050%20000%20par%20an](https://fr.talent.com/salary?job=ingenieur+informatique#:~:text=Le%20salaire%20m%C3%A9dian%20pour%20les,%E2%82%AC%2050%20000%20par%20an)

Reproduction de l’analyse quantitative

Le code permettant de reproduire les analyses quantitatives présentes dans cet article (figures et tableaux) est disponible sur le dépôt GitHub suivant : <https://github.com/gaalcaras/djcdn-data>.